

Universidade do Minho
School of Engineering

Project in Formal Methods in Programming

Final Report

MSc Software Engineering

- Cláudia Faria (PG60240)
- Patrícia Bastos (PG60287)

Supervisor: José N. Oliveira (DIUM & INESC TEC)

July 24, 2026

Contents

I	Report	3
1	Introduction	3
2	Protocol	4
2.1	Parameters and Definitions	4
2.2	Mask Update	6
2.3	Encoding	7
2.4	Robustness to packet loss	9
3	Implementation	9
4	Correctness	12
5	Discussion	14
6	Conclusion and Future Work	15
II	CCSDS Standard	16
III	Haskell Code	21
7	Data Types	21
8	ENCODER	22
9	DECODER	29
10	RLE	36
11	pocket.hs	37
12	Auxiliary functions	39
13	Basic properties	40
14	Correctness argument details	41

Formal verification of the Pocket+ lossless compression algorithm

Part I

Report

Introduction

POCKET+ is a lossless compression algorithm for spacecraft housekeeping telemetry, designed to compress fixed-length telemetry packets individually, as they are generated, using only low-level bitwise operations. This makes it lightweight enough to run on flight hardware, and its built-in robustness to packet loss makes it suitable for the unreliable communication links typical of space missions. It is the CCSDS Recommended Standard For Robust Compression of Fixed-Length Housekeeping Data [?].

Among all the advantages of this algorithm, two stand out the most: it makes compression performance more predictable for all packets, reducing important parameters such as worst compression rate and standard deviation, and it significantly reduces the amount of design work required to select telemetry acquisitions, rates, and the introduction of asynchronous events.

There already exist implementations of this protocol and articles considering the effect of flags on the standard's compression efficiency, as analyzed in [?]. But despite operating in environments where data integrity is crucial, a proof of correctness for POCKET+ is yet to be made. Proving that this compression algorithm is correct means showing that after the compression of a certain input, the output of its decompression does not differ from that input. This means that no information is lost or incorrectly reconstructed, in the case of packet losses.

This project was requested by the company Telespazio and aims to provide such proof by implementing the algorithm in Haskell, whose mathematical foundations make it the

perfect choice for translating the code into algebraic specifications. Relational algebra will then be used to assist and provide the algorithm’s correctness.

Protocol

The POCKET+ protocol is a lossless compression algorithm designed for fixed-length housekeeping data. Its goal is to efficiently encode sequences of binary input vectors by identifying redundancy between successive packets. The protocol consists of four main components:

Table 1: The core of the protocol

Component	Purpose
Flags and Parameters	Mostly user-specified, determine the protocol’s behaviour
Mask Update	Identifies predictable bits
RLE	Packet encoding function
Resynchronization	Restores information lost

Parameters and Definitions

The parameters and definitions used in the protocol are essential for understanding how the Mask Update and Encoding stages work and provide the necessary context for interpreting the operations performed by the Encoder and Decoder.

The user can choose to specify certain parameters that influence the behavior of the compressor, such as the initial mask vector and a flag that determines whether to include the entire input vector in the output. These parameters allow for flexibility and can be adjusted based on the specific requirements of the data.

The two most important parameters are always user-defined and essential for the packet loss robustness properties of the protocol.

The flag that controls the inclusion of the entire input vector in the output is particularly important, as it allows for a trade-off between compression efficiency and the ability to recover from data loss. By including the entire input vector, the compressor can ensure that all information is preserved, but it may result in larger output sizes. On the other hand, by only including the unpredictable bits, the compressor can achieve better compression ratios, but it may be more susceptible to data loss.

Similarly, the minimum required effective robustness level parameter allows the user to specify how many consecutive output vectors can be lost without affecting the ability to

decompress the data and providing a guarantee of the compressor's resilience to packet loss.

User-Specified Variables

New Mask flag (p)

Indicates whether the Mask will be replaced, enabling bit positions to change classification from unpredictable to predictable if they have not changed state since the last time.

Send Mask flag (f)

Causes the entire mask vector to be encoded in the output vector when set to one.

Uncompressed flag (r)

Causes the entire input vector to be included in the output vector when set to one.

Minimum Required Robustness Level (R)

Controls the guaranteed number of consecutive output vectors that can be lost prior to the current output vector without affecting the ability to decompress it. The effective value will always be equal to or greater than this.

Cycles

A **cycle** is a complete run of the Mask Update and Encoding operations, with the full rotation of the parameters.

Cycles are relevant in:

- **Section 5.3.2.2** — when counting consecutive occurrences of no mask changes, starting from the first cycle not covered by the minimum required effective robustness level.
- **Section 5.3.3.1** — when using the above count for OR-ing Changes.

Predictable Bits

A bit is **predictable** when, for that same index, two consecutive input vectors have the same value.

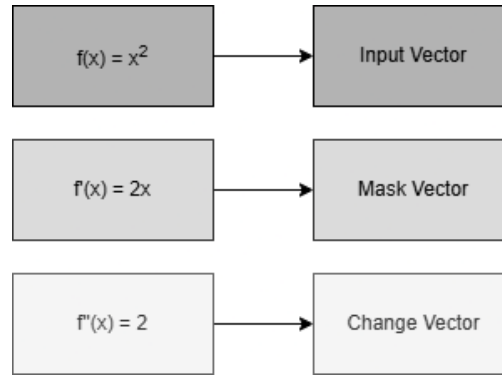


Figure 1: Mask Update "discrete derivatives"

Value	Meaning
0	The two input vectors have the <i>same</i> bit for that index.
1	The two input vectors have a <i>different</i> bit for that index.

Mask Update

The core idea of POCKET+ is that most housekeeping telemetry values don't change between successive packets.

The Mask Update operation identifies redundancy - predictable bits - between the packets. It is based on Differential pulse-code modulation (DPCM), which changes between consecutive samples of a signal, rather than the signal's value directly [?]. DPCM is then decoded by integrating DPCM samples over time.

To better understand the concept behind the Mask Update, we can think of the Input as having some sort of "discrete derivatives". The Mask Update identifies which bits of the input vector are changing, and which are not, by comparing the current input vector with the previous one. The bits that are changing are marked as unpredictable, while the bits that remain the same are marked as predictable. This information is then used in the encoding process to efficiently compress the data by only encoding the unpredictable bits. The Change vector, which indicates which bits of the mask have changed between cycles, can be thought of as a "second derivative" of the input vector, as it captures the changes in the predictability of the bits over time.

There are three main components in the Mask Update: Mask, Build, and Change. They are all calculated based on the exclusive-or between the current input vector and the previous one.

Mask

Identifies (XOR) unpredictable bits.

Compares (OR) with the last calculated difference (Mask or Build, based on user-defined p).

- **Mask bit = 0** if both the current XOR and the previous mask are 0 $\Rightarrow$ the bit has been stable across both steps.
- **Mask bit = 1** if either the current XOR or the previous mask is 1 $\Rightarrow$ the bit is, or recently was, unpredictable.

Change

Identifies which bits of the mask changed.

- **0** — the mask bit is the same as the previous mask bit; predictability status is unchanged.
- **1** — the mask bit differs from the previous mask bit; a bit has become predictable or unpredictable.

Build

Accumulates the same information as Mask in parallel, but **resets to zero when $p = 0$** , marking all positions as predictable.

This enables bit positions to change classification from unpredictable to predictable if they have not changed state since the last time the new mask flag was set to one.

Encoding

Run-length Encoding (RLE) is a common and simple form of lossless data compression in which consecutive occurrences of the same data are stored as a single data value and count. Once the unpredictable bits have been isolated, RLE is applied to encode the changes of the mask itself (our “second derivative”), which indicates which bits of the mask have changed between cycles. The Change vector will have long runs of 0s (predictable bits) that RLE will handle efficiently.

In the Encoding, the output vector is a concatenation of three variable-length binary vectors, H, Q, and U, which encode different information about the input vector and the mask. The exact content of these vectors depends on the user-specified parameters and the current state of the mask.

H

Encodes the positions in the mask vector that have changed over a number of cycles and indicates whether each change resulted in a predictable or unpredictable classification.

Specifically encodes **Mask Change + Effective Robustness Level + Help structures**.

Mask Change (X)

Run-Length Encoding applied to Change vectors OR-ed together. The amount of previous Changes considered is the *number of cycles* demanded by the current minimum required robustness level.

Effective Robustness Level (V)

Number of cycles that are included in the mask change information for this cycle.

Help Structures

e and *k*

Indicate information on the mask values for each change.

- c* Indicates if *p* was set to one more than once in the last cycles. Some bits may have transitioned from unpredictable to predictable, so those bits need to be explicitly included in U alongside the unpredictable bits.
- d* Indicates if *f* and *r* are both zero, meaning there is no requested new mask and the header is already sufficient to describe the block, so no additional flag is needed in U. Serves to indicate no dependencies from previous packets.

Q

Encodes the Mask by only transmitting it when `sendMask` is set and no new mask is incoming.

- If $f = 1$: Encodes the entire mask. XORs the mask against a shifted version of itself, computing the *difference between adjacent bits* of the mask rather than the bits

themselves, then RLE-encodes the result.

- If $f = 0$ and $r = 0$: Encodes nothing — can be decoded from the mask changes.
- If $f = 0$ and $r = 1$: Encodes no change.

U

Encodes a copy of the input vector or the unpredictable bits.

If $r = 1$ (**block is uncompressed**):

- Encodes the **length of the input vector** + full input copy.

If $r = 0$ (**block is compressed**):

- If $c = 1$: encodes the **unpredictable bits** + bits that have gone from unpredictable to predictable.
- If $c = 0$: encodes **only the unpredictable bits**.

Note: $f = 1$ adds a 0 flag at the start for header disambiguation purposes.

Robustness to packet loss

The highlight of the protocol is its robustness to packet loss, which is achieved through a resynchronization process during the decoding. When a packet is lost, the Decoder can use the information from the subsequent packets to recover the original input vector.

Focus will not be given to this component, as it is not currently implemented at the time of this report.

Implementation

There already exists a Pocket+ implementation in C++ by Vision Space [?], but it is not formally verified. Our goal is to provide a Haskell implementation that can be formally verified using relational algebra. Therefore, the implementation will be done from scratch, following the specifications provided in the standard and using the C++ implementation as a reference for testing and validation.

The first thing to be noted in this protocol is that the Encoder is a finite state machine, split into the Mask Update and the Encoding stages. The Mask Update calculates the Mask,

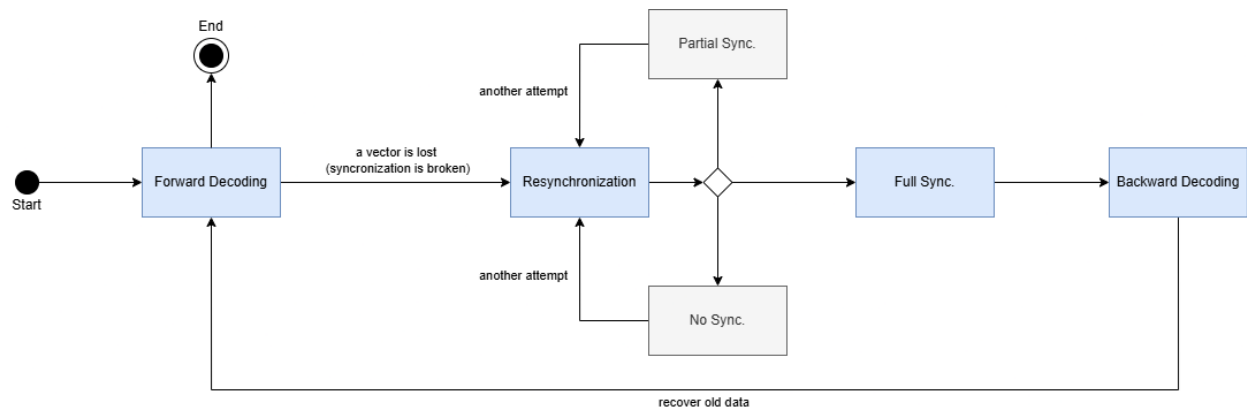


Figure 2: Decoding with packet loss

Build, and Change vectors, which make up the cycle's state, while the Encoding stage calculates the output vector based on the input vector and the compressor parameters. Essentially, at each cycle, the protocol creates a new state based on the previous state and the current input, and then produces an output based on that new state and the current input. This behaviour is that of a Mealy Machine.

A Mealy Machine is a special type of machine in finite automata theory where the output is determined by both the current state and the current input. A Mealy Machine can produce different outputs for the same state based on different inputs. It has two transition functions:

- **State transition function:** defines how the machine moves from one state to another based on the input.
- **Output transition function:** specifies the output produced for each input given to a state.

To set up our Mealy Machine, we will compose two pairs of functions. The first pair will be responsible for the state transition, while the second pair will be responsible for the yielding the output. In Haskell, it will be defined as,

$$m = \langle \pi_1, f \rangle \cdot \langle g, \pi_2 \rangle$$

where π_1 and π_2 are the projection functions that extract the new state and the output, respectively.

Essentially, the first pair $\langle \pi_1, f \rangle$ will take the current state and input and produce the new state, while the second pair $\langle g, \pi_2 \rangle$ will take the new state and input and produce the output.

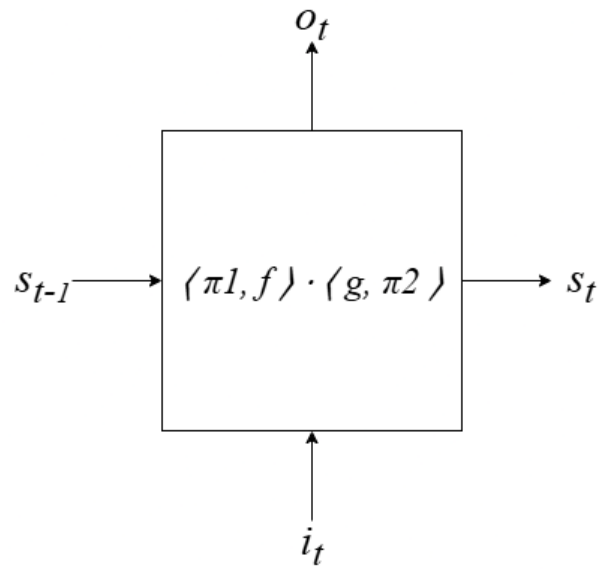


Figure 3: Mealy Machine

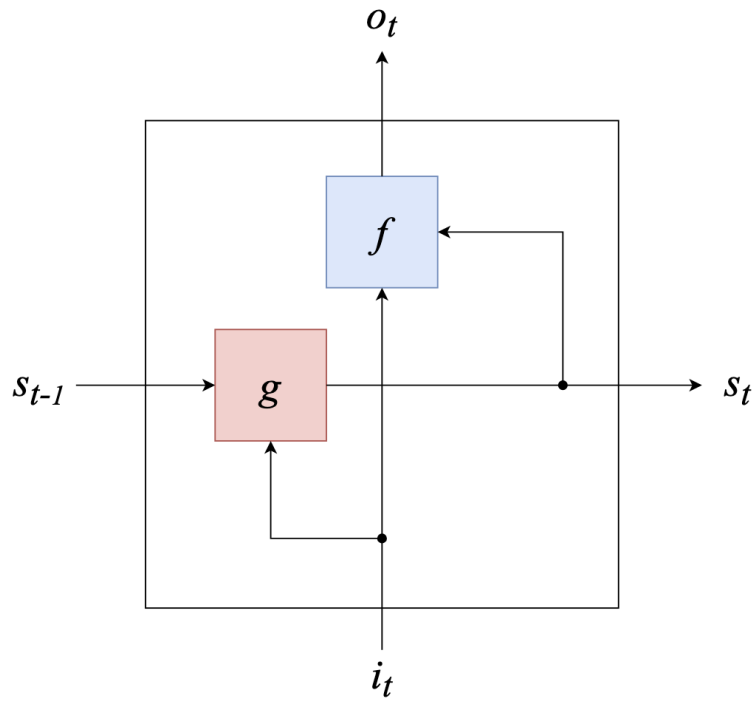


Figure 4: A Closer Look At The Mealy Machine

Each cycle of the Encoder can be seen as application of the Mealy Machine, where each state contains the Mask, Build, and Change vectors along with the last Input vector, length of the vectors, and parameters. The input is the current input vector, and the output is the concatenation of the H, Q, and U vectors.

Similarly, the Decoder will also behave as a Mealy Machine, where the state will be the same, the input will be the current output vector to be decoded, and the output will be the original recovered input vector.

Correctness

For the proofs in this report, we will be using relational algebra, which provides a powerful framework for reasoning about the properties of relations, and therefore functions, and their compositions [?]. We will make use of converses and pointfree decomposition of the main components' functions and properties to aid our proof of correction. Additional correctness details are provided at the end of this document.

The goal of the Pocket+ protocol is to ensure that the output of the Decoder is always the same as the original input vector that was fed to the Encoder. We might think of the Encoder and Decoder as two relations, where:

$$Decoder = Encoder^\circ \tag{1}$$

This is, however, too strong. For a compressing and decompressing protocol, it is enough that:

$$Decoder \cdot Encoder = id \tag{2}$$

This means that the composition of the Decoder and Encoder relations is the identity relation, which implies that for every input vector, the output of the Decoder after encoding and then decoding will be the same as the original input vector.

However, this proof is far from trivial. The Encoder and Decoder are complex functions with many components, and due to the presence of user-specified parameters and robustness adjustments, the two don't mirror each other perfectly. This is a consequence of the protocol's design, which prioritizes compression performance and robustness over simplicity. Therefore, for now, we will break down the proof into smaller components that make up the core the compression and show that each of those components is correct.

Reversibility

Lossless compression implies no information is lost during the encoding and decoding process. This makes reversibility our primary concern, which can be ensured with minimal complements [?]. Essentially, a minimal complement of a function contains just

enough information that was originally missing to make it reversible. By attaching this minimal complement, the function becomes injective, and thus reversible.

Run-length Encoding

Considering the functions' signatures, we can see that the RLE function is not injective, as *unrle* requires an integer parameter, the length of the original vector, to be able to complete the remaining zeros.

$$rle :: [Bit] \rightarrow [Bit]$$

$$unrle :: [Bit] \rightarrow Int \rightarrow [Bit]$$

However, by attaching this length as a minimal complement to the RLE function, we can make it injective and thus reversible. By implementing the run-length encoding, with minimal complements, we will ensure that the heart of the compression is reversible and correct-by-construction. Then, all that is left is uncurrying the decoding function, and we obtain the following reversible pair of functions:

$$rle' :: ([Bit] \rightarrow ([Bit], Int))$$

$$rle' = \langle rle, length \rangle$$

$$unrle' :: ([Bit], Int) \rightarrow [Bit]$$

$$unrle' = \widehat{unrle}$$

Although the recovering of the zeros is not important in the context of the protocol, it is essential for the proof of correctness, as it allows us to show that the RLE functions are inverses of each other. Naturally, this isn't done just yet. The next step is to look at the definition of the RLE functions and see how we can decompose it into smaller functions that are easier to analyze and prove correct. This proof can be found at the end of this document and ensures the correctness of this core component of compression.

Bit extraction

After run-length encoding, another important compression operation is the bit extraction, which is responsible for extracting only the unpredictable bits from the input vector. The bit extraction operation extracts the bits at positions where the mask signals a change, discarding the rest that can be predicted from the previous input vector. During the decoding process, the patch function takes the extracted bits and reinserts them into their original positions based on the mask, reconstructing the original input vector.

Together they implement the compression: the encoder sends only the unpredictable bits and the decoder puts them back into the right slots. However, this doesn't mean that they are inverses of each other. In fact, the bit extraction function is not injective, as different input vectors can yield the same extracted bits. The proof is then made even more difficult by the number of 5 cases in the encoding of the third sub-vector, which depend on the user-specified parameters.

Robustness

The components relevant to robustness are the Change vector and the effective robustness level. The Change vector identifies which bits of the mask have changed between cycles, while the effective robustness level determines how many previous cycles' changes are considered when encoding the changes in the current cycle in vector X . The effective robustness level must be equal to or greater than the minimum required effective robustness level specified by the user, which is always true, by definition of V in section 5.3.2.2, where C , which counts occurrences, is always a positive integer.

The X is then encoded in the first sub-vector of the output and necessary in the encoding of the input vector in the third-subvector, when c is 1, to encode that extra information.

The Pocket+ protocol encodes extra information at certain cycles, as specified by the user, to ensure that the decoder can recover from packet losses through the resynchronization process. However, this report does not implement the resynchronization process at this time.

Discussion

The encoder and decoder are not perfect mirrors of each other, as the protocol's design prioritizes compression performance and robustness over simplicity. Finding direct converses is not enough, and the chosen approach was to break down the proof into smaller components that make up the core of the compression and show that each of those components is correct.

The 124.0-B-1 Standard itself is not a simple read. The presence of user-specified parameters and robustness adjustments makes the implementation and proof of correctness more difficult. Having struggled with the standard, we hoped to create a simpler but richer formal specification of the protocol, complete with descriptive diagrams to aid in comprehension. This specification can be found at the end of this document, along with the respective code, and we hope it can be used as a reference for future work on the protocol.

Haskell was chosen to implement the protocol due to its strong type system and mathematical foundations, which make it easier to translate the code into algebraic specifica-

tions and reason about its properties. Even if the reader is not fluent in Haskell, the code, equipped with diagrams, should be easy to understand and follow. With Haskell, we translated the Mealy Machine structure of the protocol into a composition of two pairs of functions, one for the state transition and another for the output generation. This structure allowed us to reason about the protocol’s behavior in a modular way, analyzing the state transitions and output generation separately.

Conclusion and Future Work

The protocol implementation started in February 2025, as part of the final project of the Bachelor’s in Computer Science. In June 2026, now under the final project of Formal Methods of Programming in the Master’s in Software Engineering, the implementation was completed, and the proof of correctness started. The proof is still work in progress, with focus on the main components of the protocol, such as the RLE and bit extraction functions, and the robustness properties. The final goal is to show that the composition of the Decoder and Encoder relations is the identity relation, proving the correctness of the protocol.

Currently, the next step of this implementation is cross-validation for compliance with the four corpora of housekeeping telemetry data defined by the CCSDS Data Compression working group. Future work regarding the proof of correctness includes completing the existing proofs and further exploring the role of the user-specified parameters in the protocol’s behavior and correctness. Additionally, the resynchronization process can be implemented, as a crucial step to the conclusion of this implementation.

This report contributes with a richer formal specification of the protocol, an implementation in Haskell to be used as a reference for testing and validation, and a proof of correctness for the main components of the protocol, which are essential for its correctness, using relational algebra.

This work lays the foundation for a complete proof of correctness for the Pocket+ protocol, and perhaps encourage further research into the formal verification of similar algorithms with critical applications, where correctness is of utmost importance.

References

- [1] Consultative Committee for Space Data Systems. Robust compression of fixed-length housekeeping data. Recommended Standard CCSDS 124.0-B-1, CCSDS, 2023.
- [2] Miguel Hernández-Cabronero, David Evans, Joan Bartrina-Rapesta, Francesc Aulí-Llinàs, Ian Blanes, and Joan Serra-Sagristà. Resiliency and Efficiency of the CCSDS 124.0-B-1 Telemetry Compression Standard. *IEEE Access*, 12:36702–36711, 2024.

- [3] MathWorks. Differential pulse code modulation, 2026. <https://www.mathworks.com/help/comm/ug/differential-pulse-code-modulation.html>.
- [4] A. Neri, R.S Barbosa, and J.N. Oliveira. Compiling quantamorphisms for the ibm q experience. *IEEE Transactions on Software Engineering*, 48(11):3–9, 2022.
- [5] J. N. Oliveira. Program Design by Calculation, 2024. Unpublished textbook draft, Sep. 2024 (313 p.). Informatics Dept., U.Minho (PDF).
- [6] Vision Space. Pocketplus, 2023. <https://github.com/visionspacetec/PocketPlus>.

Part II

CCSDS Standard

3 INPUTS AND PARAMETERS

3.1 OVERVIEW

The compressor losslessly encodes a sequence of length- F binary input vectors $\mathbf{I}_t$, $t = 0, 1, \dots$. At each time index t , the compressor produces a variable-length binary output vector $\mathbf{o}_t$ that losslessly encodes $\mathbf{I}_t$. This encoding depends on several user-specified variables defined in 3.2:

- The initial mask vector, $\mathbf{M}_0$, is a length- F binary vector used to initialize the mask.
- The minimum required effective robustness level, R , controls the guaranteed number of consecutive output vectors that can be lost prior to the current output vector without affecting the ability to decompress it. The effective robustness level will be equal or greater than this value.
- The new mask flag, $\dot{p}_t$, when set to one, causes the current mask vector to be replaced with the mask being built. This enables bit positions to change classification from unpredictable to predictable if they have not changed state since the last time the new mask flag was set to one.
- The uncompressed flag, $\dot{r}_t$, causes the entire input vector to be included in the output vector when it is set to one; otherwise, only those bits classified as unpredictable are included.
- The send mask flag, $\dot{f}_t$ causes the entire mask vector to be encoded in the output vector when it is set to one. All compressor parameters needed for decompression can be determined from the output binary vectors.

3.2 INPUT

At each time index $t = 0, 1, \dots$, the input to the compressor shall be a length- F binary input vector $\mathbf{I}_t$, where the input vector length F shall be a user-specified integer in the range $1 \leq F \leq 2^{16} - 1$.

3.3 PARAMETERS

3.3.1

The user-specified initial mask vector, $\mathbf{M}_0$, shall be a length- F binary vector.

NOTE — $\mathbf{M}_0$ could be derived from analysis of previous mask values or a stored value of the mask from the last time this was run. Setting $\mathbf{M}_0 = 0$ (i.e., all positions predictable) is often a reasonable default.

3.3.2

In addition to the initial mask vector, compression parameters at each time index t consist of the following:

- a) The user-specified minimum required effective robustness level, R , which shall be an integer between 0 and 7 at each time index t .
- b) The user-specified new mask flag, $\dot{p}_t$, which shall be zero or one at each time index t .
- c) The send mask flag, $\dot{f}_t$, which shall be $\dot{f}_t = 1$ for $t \leq Rt$; otherwise, $\dot{f}_t$ is user-specified and shall be zero or one.
- d) The uncompressed flag, $\dot{r}_t$, which shall be $\dot{r}_t = 1$ if $t \leq Rt$; otherwise $\dot{r}_t$ is user-specified and shall be zero or one.

NOTE — The values of user-specified parameters $\mathbf{M}_0$, R , t , $\dot{f}_t$, $\dot{p}_t$, and $\dot{r}_t$ need not be known in advance, or communicated via means external to the compressor, for successful decompression.

4 MASK UPDATE

4.1 OVERVIEW

This section specifies the compressor's mask update stage. Two binary vectors, $\mathbf{M}_t$ (the mask) and $\mathbf{B}_t$ (the build), are updated in parallel based on the exclusive-or between the current input binary vector and the previous one (see 4.2.1 and 4.2.2).

Since no such difference is defined for the first input vector, build is initialized to a zero vector, that is, $\mathbf{B}_0 = 0$. For subsequent input binary vectors, a bitwise XOR of sequential input binary vectors is used to update mask and build. The change vector, $\mathbf{D}_t$ is computed as the exclusive-or between the current mask and previous mask (see 4.2.3).

When the new mask flag, $\dot{p}_t$, is set to one, the mask is replaced by build, and build is reset to a zero vector; that is, all positions are classified as predictable (see 3.3.2). Hence, even though the update processes for mask and build depend on the same calculations, these vectors are not in general identical because they may be reset at different times during the encoding process.

4.2 MASK UPDATE

4.2.1

For $t \geq 0$, the build vector is a length-F binary vector defined as

$$\mathbf{B}_t = \begin{cases} (\mathbf{I}_t \text{ XOR } \mathbf{I}_{t-1}) \text{ OR } \mathbf{B}_{t-1}, & t > 0, \dot{p}_t = 0 \\ 0, & \text{otherwise} \end{cases}, \quad (3)$$

where $\dot{p}_t$ is defined in 3.3.2.

4.2.2

For $t > 0$, the mask vector is a length-F binary vector defined as

$$\mathbf{M}_t = \begin{cases} (\mathbf{I}_t \text{ XOR } \mathbf{I}_{t-1}) \text{ OR } \mathbf{M}_{t-1}, & \dot{p}_t = 0 \\ (\mathbf{I}_t \text{ XOR } T(t-1)) \text{ OR } \mathbf{B}_{t-1}, & \text{otherwise} \end{cases}, \quad (4)$$

where $\dot{p}_t$ is defined in 3.3.2.

NOTE — At $t = 0$, the initial mask vector $\mathbf{M}_0$ is user-specified (see 3.3.1).

4.2.3

For $t \geq 0$, the change vector $\mathbf{D}_t$ is a length-F binary vector defined as

$$\mathbf{D}_t = \begin{cases} \mathbf{M}_t \text{ XOR } \mathbf{M}_{t-1}, & t > 0 \\ 0, & \text{otherwise} \end{cases}. \quad (5)$$

5 ENCODER

5.1 OVERVIEW

This section specifies the encoding stage of the compressor. Subsection 5.2 defines functions used in the encoding process, and 5.3 specifies the calculation of the output vector from the input vector and compressor parameters.

As specified in 5.3, at any time, t , the encoder output, is a variable-length binary vector, $\mathbf{o}_t$, that is a concatenation of three variable-length binary vectors, $\mathbf{h}_t$, $\mathbf{q}_t$, and $\mathbf{u}_t$ described below.

The first binary vector, $\mathbf{h}_t$, encodes information about recent mask changes (see 5.3.3.1). The most recent change vector is ORed with previous change vectors (to provide robustness against packet loss), and the result is run-length encoded. Next, the effective robustness level, V_t , is encoded using 4 bits, followed by information on the mask values for each change. This is followed by the value of $\mathbf{c}_t$, which indicates if $\dot{p}_v$ was set to one more than once in the period covering this cycle and the previous V_t cycles. This information is used in the encoding. The final bit indicates whether user-specified parameters $\dot{f}_t$ and $\dot{r}_t$ are both zero. If so, the values of those parameters are not encoded in $\mathbf{q}_t$ or $\mathbf{u}_t$.

The second binary vector, $\mathbf{q}_t$, encodes information about the entire mask (see 5.3.3.2). Although mask changes alone would be sufficient to reconstruct the mask, the option to send the entire mask can be requested by setting the send mask flag, $\dot{f}_t$, to one (see 3.3.2). In this case, $\mathbf{q}_t$ consists of a '1' concatenated with the mask that has been preprocessed and run-length encoded.

Finally, the third binary vector, $\mathbf{u}_t$, encodes different information depending on the values of $\dot{r}_t$ and $\mathbf{c}_t$. If $\dot{r}_t$ is one (see 3.3.2), then $\mathbf{u}_t$ always contains a bit string that encodes the value of the input block length E , followed by the input vector. Otherwise, if $\mathbf{c}_t$ is one, then $\mathbf{u}_t$ contains the unpredictable bit values and the values of any bits that changed from unpredictable to predictable in the present change vector or the previous V_t change vectors. If $\mathbf{c}_t$ is not one, then $\mathbf{u}_t$ contains only the unpredictable bit values. In all cases, the output is preceded by $\dot{r}_t$ if its value was not already specified by the last bit of $\mathbf{h}_t$.

5.2 BASIC ENCODING FUNCTIONS

5.2.1 OVERVIEW

This subsection defines the counter encoding, run-length encoding, and bit extraction functions used by the encoding procedure specified in Section 5.3.3.

5.2.2 COUNTER ENCODING FUNCTION

Given a positive integer $1 \leq A \leq 2^{16} - 1$, the counter encoding function, denoted $\text{COUNT}(A)$, maps A onto a variable-length binary vector following Table 2.

Table 2: Counter Encoding Table	
Input integer, A	Output vector
$A = 1$	0
$2 \leq A \leq 33$	110 $\text{BIT}_5(A - 2)$
$A \geq 34$	111 $\text{BIT}_E(A - 2)$

E is calculated as

$$E = 2\lceil \log_2(A - 2) + 1 \rceil - 6. \quad (9)$$

Note: The equation above calculates a bit string length consisting of a number of zeros concatenated with a minimum-length bit string encoding of the integer $(A - 2)$. As the relationship between the number of

preceding zeros and the length of the bit string encoding ($A-2$) is unique, it can be used by the decompressor to parse the output.

5.2.3 RUN-LENGTH ENCODING

Given a binary vector $\mathbf{a}$, the run-length encoding of $\mathbf{a}$, denoted $\text{RLE}(\mathbf{a})$, is a variable-length binary vector defined as

$$\text{RLE}(\mathbf{a}) = \text{COUNT}(C_0) \mid \cdots \mid \text{COUNT}(C_{H(\mathbf{a})-1}) \mid 10, \quad (10)$$

where C_i is one more than the number of consecutive 0 bits preceding the i -th 1 bit in $\mathbf{a}$, starting at the MSB and decreasing.

Notes:

1. If the vector $\mathbf{a}$ ends with one or more zeros, they are not explicitly encoded via the COUNT function, as they can be inferred from the length of the input vector and the number of 1 bits, $H(\mathbf{a})$.
2. When vector $\mathbf{a}$ does not contain any '1' bits, the COUNT function will yield a 10 bit string.
3. Figure 5 provides an illustration of the run-to-count encoding.

0001 000001 000001 00001 0000001 1001 0000000000000000

$$C_0 = 4, \quad C_1 = 6, \quad C_2 = 1, \quad C_3 = 6, \quad C_4 = 5, \quad C_5 = 7, \quad C_{H(\mathbf{a})-1} = 3$$

Figure 5: Example of the Run-to-Count Encoding, converting a binary vector $\mathbf{a}$ into a sequence of integers.

5.2.4 BIT EXTRACTION FUNCTION

Given two binary vectors $\mathbf{a}$, $\mathbf{b}$ having the same length, the bit extraction of $\mathbf{a}$ relative to $\mathbf{b}$, denoted $\text{BE}(\mathbf{a}, \mathbf{b})$, is the sequence of bits in $\mathbf{a}$ taken from the positions where $\mathbf{b}$ has a 1 bit. That is,

$$\text{BE}(\mathbf{a}, \mathbf{b}) = \dot{a}_{g_{H(\mathbf{b})-1}} \mid \cdots \mid \dot{a}_{g_0}, \quad (11)$$

where g_i denotes the position of the i -th 1 bit in $\mathbf{b}$, starting from the MSB.

5.3 ENCODING STEP

5.3.1 OUTPUT BINARY VECTOR STRUCTURE

The encoder output is comprised of the sequence of variable-length binary output vectors $\mathbf{o}_0, \mathbf{o}_1, \mathbf{o}_2, \mathbf{o}_3, \dots$. Each output binary vector $\mathbf{o}_t$ shall be defined as

$$\mathbf{o}_t = \mathbf{h}_t \mid \mathbf{q}_t \mid \mathbf{u}_t, \quad (12)$$

where variable-length binary vectors $\mathbf{h}_t$, $\mathbf{q}_t$, and $\mathbf{u}_t$ are specified in Sections 5.3.3.1, 5.3.3.2, and 5.3.3.3, respectively.

5.3.2 INTERMEDIATE CALCULATIONS

5.3.2.1 For $t \geq 0$, $\dot{d}_t$ is defined as

$$\dot{d}_t = \begin{cases} 1, & \text{if } \dot{f}_t = 0 \text{ and } \dot{r}_t = 0, \\ 0, & \text{otherwise,} \end{cases} \quad (13)$$

where $\dot{f}_t$ and $\dot{r}_t$ are defined in Section 3.3.2.

5.3.2.2 For $t \geq 0$, V_t is defined as

$$V_t = \begin{cases} R_t, & \text{if } (t - R_t) \leq 0, \\ R_t + C_t, & \text{otherwise,} \end{cases} \quad (14)$$

where R_t is defined in Section 3.3.2, and C_t is the maximum integer where $C_t \leq \min(t, 15) - R_t$, for which at a given time instant t , all $\mathbf{D}_{t'} = \mathbf{0}$ (see Section 4.2.3) for $t' \in \{t - R_t - 1, t - R_t - 2, \dots, t - R_t - C_t\}$.

Note: C_t counts the number of consecutive occurrences of no mask changes, starting from the first cycle not covered by the minimum required effective robustness level and working backwards in time.

5.3.3 OUTPUT VECTOR COMPONENTS

5.3.3.1 The binary vector $\mathbf{h}_t$ is defined as

$$\mathbf{h}_t = \text{RLE}(\mathbf{X}_t) \mid \text{BIT}_4(V_t) \mid \mathbf{e}_t \mid \mathbf{k}_t \mid \mathbf{c}_t \mid \dot{d}_t, \quad (15)$$

where V_t is defined in Section 5.3.2.2, $\dot{d}_t$ is defined in Section 5.3.2.1, and $\mathbf{X}_t$, $\mathbf{e}_t$, $\mathbf{c}_t$, and $\mathbf{k}_t$ are defined as follows.

$$\mathbf{X}_t = \begin{cases} \langle \mathbf{D}_t \rangle, & \text{if } R_t = 0, \\ \langle \mathbf{D}_1 \text{ OR } \mathbf{D}_2 \text{ OR } \dots \text{ OR } \mathbf{D}_t \rangle, & \text{if } (t - R_t) \leq 0, \\ \langle \mathbf{D}_{t-R_t} \text{ OR } \mathbf{D}_{t-R_t+1} \text{ OR } \dots \text{ OR } \mathbf{D}_t \rangle, & \text{otherwise,} \end{cases} \quad (16)$$

where $\mathbf{D}_t$ is the change binary vector defined in Section 4.2.3, and R_t is defined in Section 3.3.2.

$$\mathbf{y}_t = \text{BE}(\langle \neg \mathbf{M}_t \rangle, \mathbf{X}_t), \quad (17)$$

$$\mathbf{e}_t = \begin{cases} \emptyset, & \text{if } V_t = 0 \text{ or } \mathbf{X}_t = \mathbf{0}, \\ 0, & \text{if } \mathbf{y}_t = \mathbf{0} \text{ and } V_t > 0 \text{ and } \mathbf{X}_t \neq \mathbf{0}, \\ 1, & \text{otherwise,} \end{cases} \quad (18)$$

$$\mathbf{k}_t = \begin{cases} \emptyset, & \text{if } V_t = 0 \text{ or } \mathbf{X}_t = \mathbf{0} \text{ or } \mathbf{y}_t = \mathbf{0}, \\ \mathbf{y}_t, & \text{otherwise,} \end{cases} \quad (19)$$

$$\mathbf{c}_t = \begin{cases} \emptyset, & \text{if } \mathbf{k}_t = \emptyset, \\ 0, & \text{if } \mathbf{k}_t \neq \emptyset \text{ and } p_i \text{ is not set to one more than once,} \\ & \text{for } i \in \{\max(0, t - V_t), \max(0, t - V_t) + 1, \dots, t - 1, t\}, \\ 1, & \text{otherwise.} \end{cases} \quad (20)$$

5.3.3.2 The binary vector $\mathbf{q}_t$ is defined as

$$\mathbf{q}_t = \begin{cases} \emptyset, & \text{if } \dot{d}_t = 1, \\ 1 \mid \text{RLE}(\langle \mathbf{M}_t \text{ XOR } (\mathbf{M}_t \ll) \rangle), & \text{if } \dot{f}_t = 1, \\ 0, & \text{otherwise,} \end{cases} \quad (21)$$

where $\dot{f}_t$ is defined in Section 3.3.2, and $\dot{d}_t$ is defined in Section 5.3.2.1.

5.3.3.3 The binary vector $\mathbf{u}_t$ is defined as

$$\mathbf{u}_t = \begin{cases} \text{BE}(\mathbf{I}_t, (\langle \mathbf{X}_t \rangle \text{ OR } \mathbf{M}_t)), & \text{if } \dot{d}_t = 1 \text{ and } \mathbf{c}_t \neq 1, \\ \text{BE}(\mathbf{I}_t, \mathbf{M}_t), & \text{if } \dot{r}_t = 1, \\ 1 \mid \text{COUNT}(F) \mid \mathbf{I}_t, & \text{if } \dot{r}_t = 1, \\ 0 \mid \text{BE}(\mathbf{I}_t, (\langle \mathbf{X}_t \rangle \text{ OR } \mathbf{M}_t)), & \text{if } \dot{r}_t = 0 \text{ and } \dot{f}_t = 1 \text{ and } \mathbf{c}_t = 1, \\ 0 \mid \text{BE}(\mathbf{I}_t, \mathbf{M}_t), & \text{otherwise,} \end{cases} \quad (22)$$

where $\dot{r}_t$ and $\dot{f}_t$ are defined in Section 3.3.2, $\mathbf{c}_t$ is defined in Section 5.3.3.1, and $\dot{d}_t$ is defined in Section 5.3.2.1.

Part III

Haskell Code

Data Types

Section 3.2 Input

The most basic data type is

```
type Bit = Int
```

The main data types are the input vector to be compressed,

```
type I = [Bit]
```

the compressed output vector,

```
type O = [Bit]
```

and the state:

```
type S = (I, B, M, [D], Len, Param)
```

The state is a tuple of the current input vector, the build vector, the mask vector, a list of change vectors, the length of the input vector, and the parameters for the current cycle.

Section 3.3 Parameters

The initial parameters are composed of the initial mask vector, the minimum required effective robustness level, and flags. These will be updated in each cycle according to the protocol's specifications, and therefore are included in the state.

Each state carries the input, the three Mask Update vectors,

```
type B = [Bit]  -- Build vector
type M = [Bit]  -- Mask vector
type D = [Bit]  -- Change vector
```

the length of the input,

```
type Len = Int
```

and the parameters for the current round. Respectively, initial mask vector, the minimum required effective robustness level, the new mask flag, the uncompressed flag, and the send mask flag are defined as follows:

```

data Param = Param {
  M0 :: [Bit] -- M0
  , robustnessLevel :: [Bit] -- Rt (r)
  , newMask :: [Bit] -- pt
  , uncompressed :: [Bit] -- rt
  , sendMask :: [Bit] -- ft
} deriving (Eq, Ord, Show)

```

The initialization parameters¹ are

```

initParamEnc = Param
  (nzeros 32)
  [0, 2, 2, 2, 2, 2, 2, 2]
  [0, 1, 0, 0, 0, 0, 0, 0]
  [0, 1, 1, 1, 0, 0, 0, 0]
  [0, 1, 1, 1, 0, 0, 0, 0]

```

Because the parameters are updated at the start of each cycle, the first round will not be considered and are all set at 0 to align with the parameter rotation.

The initial state is:

```

initState = ([], [], [], [], 32, initParamEnc)

```

Used in verifying if there will be a new mask:

```

type QtMask = Maybe M

```

```

type V = Bit
type X = [Bit]
type Y = [Bit]
type E = [Bit]
type K = [Bit]
type C = [Bit]

```

ENCODER

The encoder has type

```

encoder :: [I] → (S, [O])

```

and is defined by

```

encoder = stream_proc gEnc fEnc initState

```

where

$$\text{stream_proc } g f = \text{mapAccumL } \widehat{mealy} \widehat{g} \widehat{f}$$

is the corresponding map-accumulate process. Its two parameters g_{Enc} and f_{Enc} define a Mealy machine:²

¹Taken from [?].

²Note concerning naming, recalling Fig. 4: g -functions have to do with updating the state while f -functions have to do with the production of output.

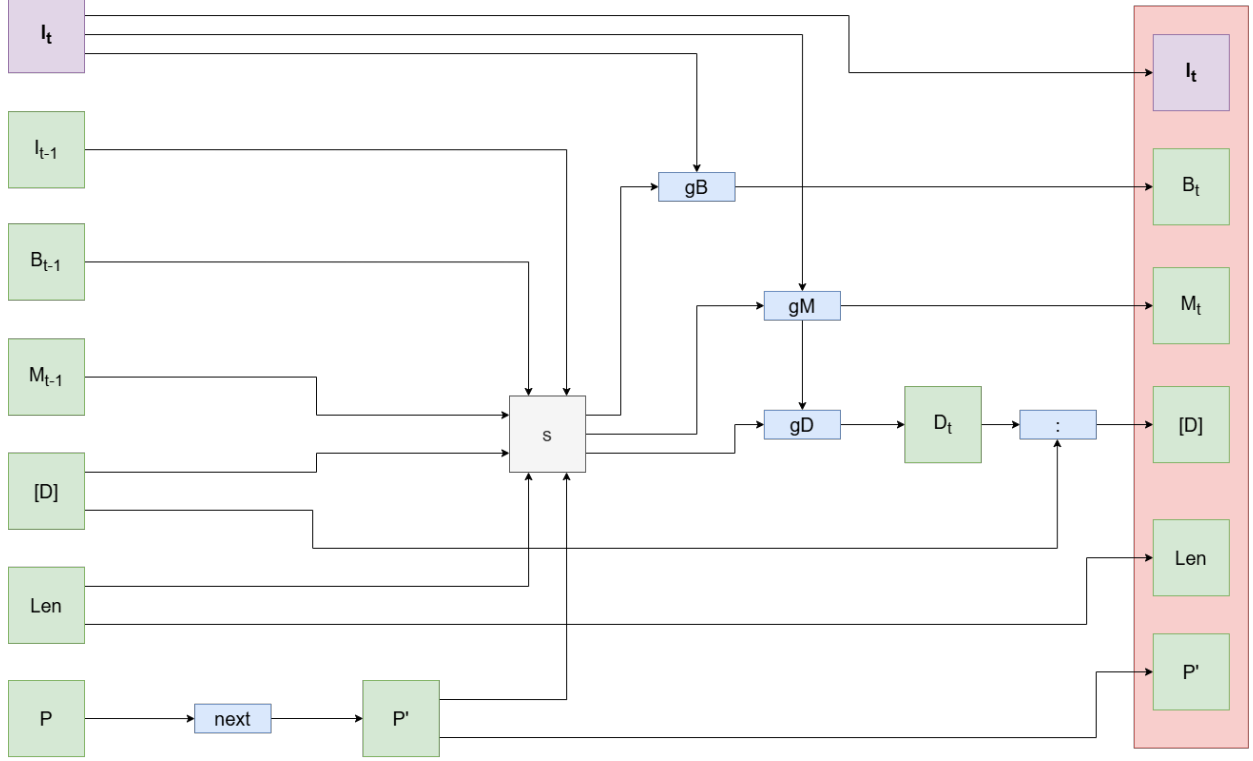


Figure 6: Encoder state update function g_{Enc} .

- $g_{Enc} :: S \rightarrow I \rightarrow S$ — updates the state
- $f_{Enc} :: S \rightarrow I \rightarrow O$ — produces the output vector

Thus `mapAccumL` iterates such a Mealy machine as much as required by the input stream.

The state update goes as follows (see Fig. 6, details in the sequel):

```

 $g_{Enc} :: S \rightarrow I \rightarrow S$ 
 $g_{Enc} (is, b, m, d, len, param) i = (i, b', m', d' : d, len, param')$ 
where
   $b' = g_B i s$ 
   $m' = g_M i s$ 
   $d' = g_D m' s$ 
   $s = (is, b, m, d, len, param')$ 
   $param' = next param$ 

```

The output vector is produced as follows (per section 5.3.1, details in the sequel):

```

 $f_{Enc} :: S \rightarrow I \rightarrow O$ 
 $f_{Enc} s i = new_H \# new_Q \# new_U$  where
   $new_H = f_H x v e k c d$ 
   $new_Q = f_Q s d$ 
   $new_U = f_U i s x c d$ 
   $x = vec_X s$ 
   $v = func_V s$ 
   $e = vec_E y x v$ 

```

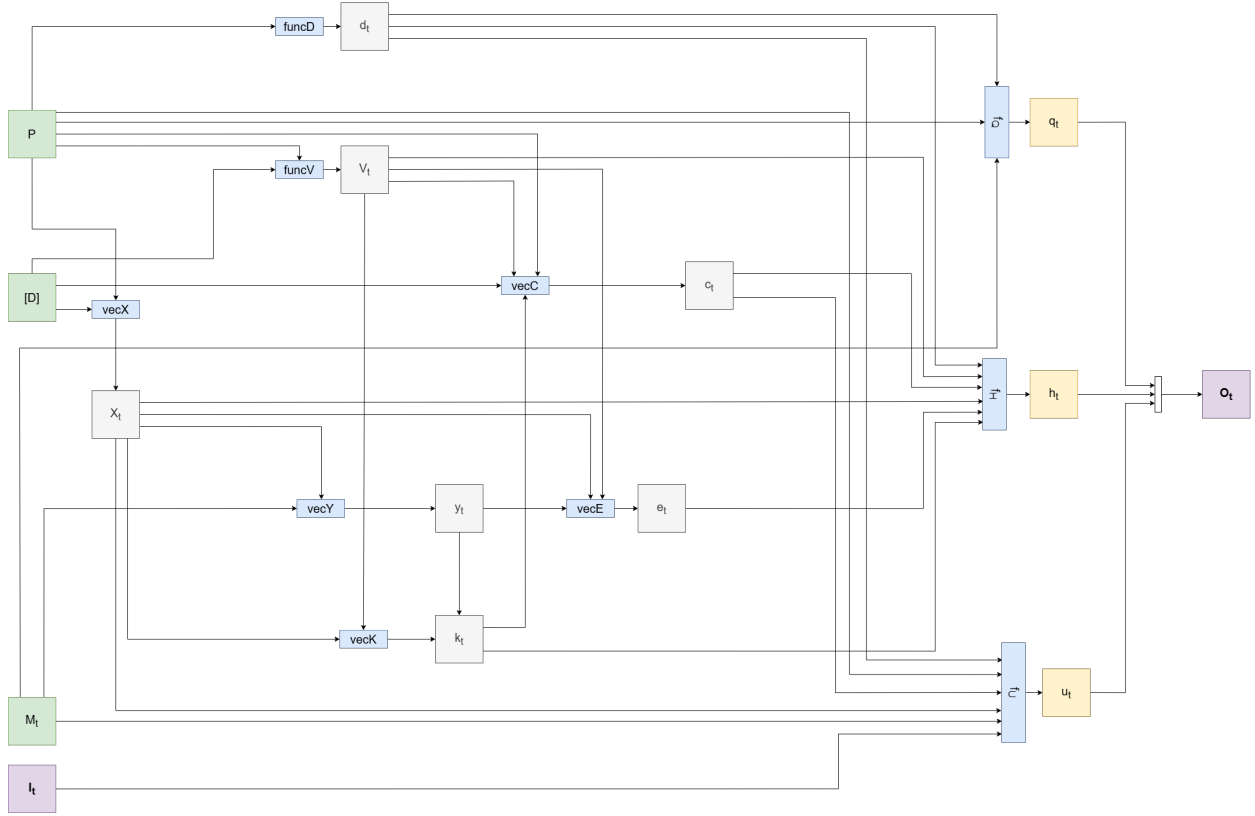


Figure 7: fEnc

$$\begin{aligned}
 k &= \text{vec}_K \, v \, x \, y \\
 c &= \text{vec}_C \, k \, v \, s \\
 \hat{d} &= \text{func}_D \, s \\
 y &= \text{vec}_Y \, s \, x
 \end{aligned}$$

Section 4.2 Mask Update

The state update by g_{Enc} is factored over three functions, each handling the corresponding component of the state, namely:

Section 4.2.1

The build,

$$\begin{aligned}
 g_B &:: I \rightarrow S \rightarrow B \\
 g_B \, i \, ([], b, -, -, -, param) &= \text{zeros} \, (M_0 \, param) \\
 g_B \, i \, (i', b, -, -, -, param) &= \text{if } a \equiv 0 \text{ then } \text{kern } i \, i' \, b \text{ else } \text{zeros } b \\
 &\quad \text{where } (a: _) = \text{newMask } param
 \end{aligned}$$

Section 4.2.2

The mask,

```

gM :: I → S → M
gM i ([ ], -, -, -, -, param) = M0 param
gM i (i', b, m, -, -, param) = kern i i' (if a ≡ 0 then m else b)
where
  (a: -) = newMask param

```

Section 4.2.3

The change:

```

gD :: M → S → D
gD m1 ([ ], -, m, [ ], -, -) = zeros m1
gD m1 (-, -, m, -, -, -) = unpredict m1 m

```

The followig function captures de essence ('kernel') of the update process:

```

kern :: Bits d ⇒ [d] → [d] → [d] → [d]
kern = zipWith3 g where g i j b = (i ⊕ j) ∨ b

```

Section 5.3.3 Output Vector Components

Concerning f_{Enc} , the output vector is produced by

Section 5.3.3.1

```

fH :: X → Bit → E → K → C → Bit → [Bit]
fH x v e k c ḋ = rle x ⊕ toNBits 4 v ⊕ e ⊕ k ⊕ c ⊕ [ḋ]

```

together with

Section 5.3.3.2

```

fQ :: S → Bit → [Bit]
fQ (-, -, m, -, -, param) ḋ
  | ḋ ≡ 1 = [ ]
  | sm ≡ 1 = 1 : rle (reverse (undpcm m))
  | otherwise = [0]
where
  (sm: -) = sendMask param

```

and

Section 5.3.3.3

```

fU :: I → S → X → C → Bit → [Bit]
fU i (-, -, m, -, -, param) x c ḋ
  | ḋ ≡ 1 ∧ c ≡ [1] = f y
  | ḋ ≡ 1 ∧ c ≠ [1] = f m
  | uncpr ≡ 1 = 1 : count (length i) ⊕ i

```

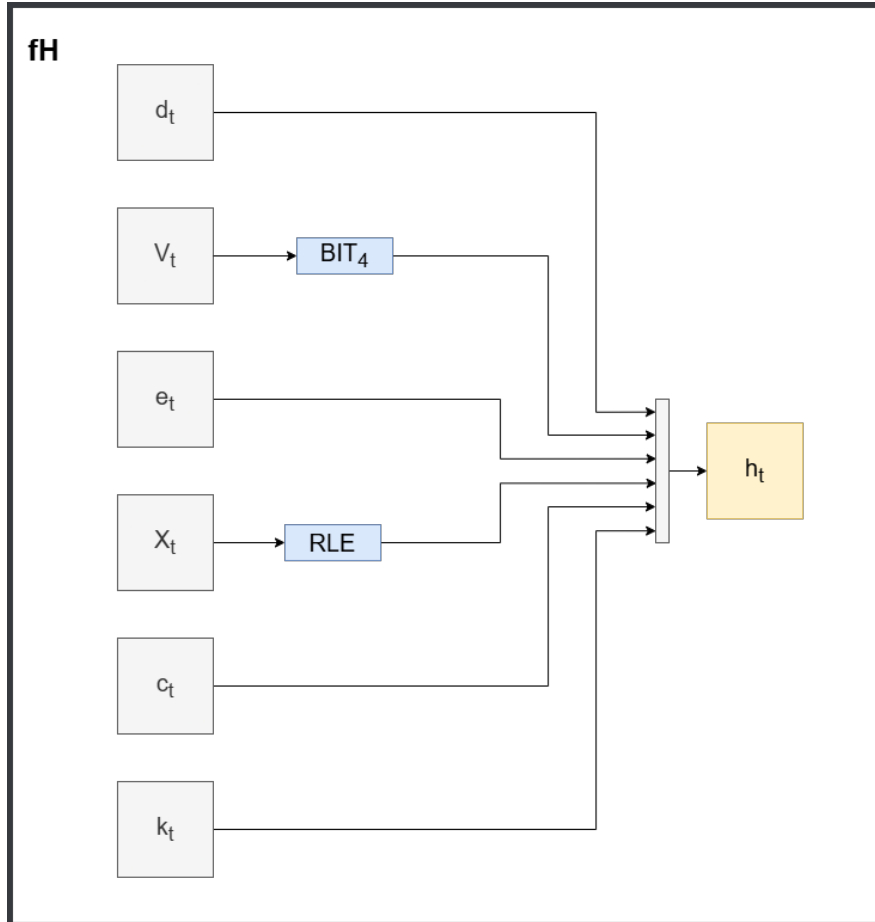


Figure 8: fH

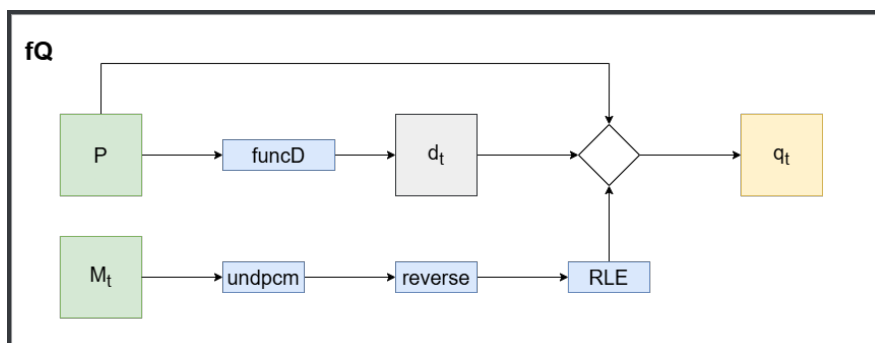


Figure 9: fQ

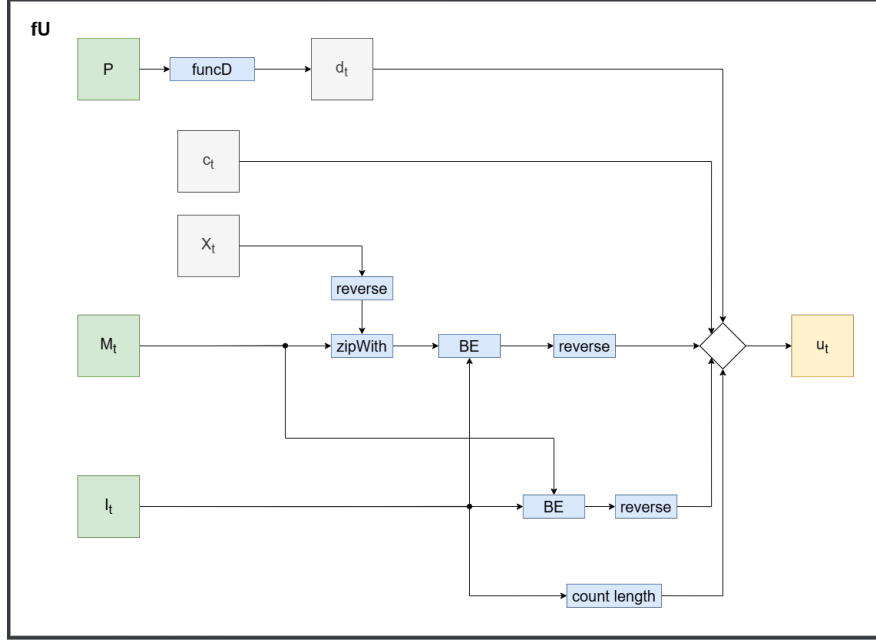


Figure 10: fU

$| \text{uncpr} \equiv 0 \wedge \text{sm} \equiv 1 \wedge \text{ch} \equiv 1 = 0 : f \ y$
 $| \text{otherwise} = 0 : f \ m$

where

$y = \text{zipWith } (\vee) (\text{reverse } x) \ m$
 $(\text{ch}: _) = c$
 $(\text{uncpr}: _) = \text{uncompressed param}$
 $(\text{sm}: _) = \text{sendMask param}$
 $f = \text{reverse} \cdot (\text{be } i)$

Section 5.2.4 Bit Extraction Function

The following function, used above, is such that $\text{be } x \ m$ keeps the a s in x that match the 'mask' M :

$\text{be} :: [a] \rightarrow [Bit] \rightarrow [a]$
 $\text{be } [] _ = []$
 $\text{be } _ [] = []$
 $\text{be } (x : a) (1 : b) = x : \text{be } a \ b$
 $\text{be } (_ : a) (0 : b) = \text{be } a \ b$

A version better for calculations is:

$\text{be} :: [a] \rightarrow [Bit] \rightarrow [a]$
 $\text{be} = \text{fmap } f \cdot \text{zip} \text{ where } f = \text{map } \text{fst} \cdot \text{filter } ((\neq 0) \cdot \text{snd})$

Section 5.3.2 Intermediate calculations

The following intermediate calculations are used in the production of the output vector, and are defined as follows.

Section 5.3.2.1

```

funcD :: S → Bit
funcD (→, →, →, →, →, param)
  | (sm ≡ 0 ∧ uncpr ≡ 0) = 1
  | otherwise = 0
where (sm: _) = sendMask param
        (uncpr: _) = uncompressed param

```

Section 5.3.2.2

```

funcV :: S → Bit
funcV (→, →, →, ds, →, param)
  | t ≤ rl = rl
  | otherwise = rl + ct
where
  (rl: _) = robustnessLevel param
  t = length ds - 1
  upper = min t 15
  x = take (upper - rl) (drop (rl + 1) ds)
  y = takeWhile (all (≡ 0)) x
  ct = length y

```

Section 5.3.3.1

For the first output sub-vector, a vector x is defined as the OR of the change vectors in the period covering the current cycle and the previous V_t cycles

```

vecX :: S → X
vecX = reverse · aux
where
  aux (→, →, →, ds, →, param)
    | rl ≡ 0 = d
    | tl - rl ≤ 0 = foldl1 l ds
    | otherwise = foldl1 l dt
  where
    (d : restD) = ds
    tl = length ds - 1
    (rl: _) = robustnessLevel param
    dt = take (rl + 1) ds
    l = zipWith (∨)

```

which is concatenated with the following functions for the calculation of the h -component of the output vector,

```

vecY :: S → X → Y
vecY (→, →, m, →, →, →) x = be x (reverse (bitflip m))
where
  bitflip = map (1-)

vecE :: Y → X → V → E
vecE y x v

```

```

|  $v \equiv 0 \vee x \equiv \text{zeros } x = []$ 
|  $y \equiv \text{zeros } y \wedge v > 0 \wedge x \not\equiv \text{zeros } x = [0]$ 
|  $\text{otherwise} = [1]$ 

```

```

vecK :: V → X → Y → K
vecK v x y
|  $v \equiv 0 \vee x \equiv \text{zeros } x \vee y \equiv \text{zeros } y = []$ 
|  $\text{otherwise} = y$ 

```

```

vecC :: K → V → S → C
vecC [] _ _ = []
vecC k v (→, →, →, ds, →, param)
|  $\text{sum } (is) > 1 = [0]$ 
|  $\text{otherwise} = [1]$ 
where
  t = length ds - 1
  is = [(max 0 (t - v)) .. t]

```

DECODER

The decoder has type:

```
decoder :: [O] → (S, [I])
```

We are currently only implementing forward decoding. The state is kept the same as the *encoder's*.

```
type S = ([I], B, M, [D], Len, Param)
```

B stays empty in decoding. Then:

```

decoder :: [O] → (S, [I])
decoder = stream.proc gDec fDec initState
where
  initState = ([], [], [], [], 0, initParamDec)

```

The order is: mask, robustness level, new mask, uncompressed flag, send mask flag. All are initialized to *null* for decoding, as we will reconstruct them from the encoded input:

```
initParamDec = Param [] [] [] [] []
```

g_{Dec} consumes previous state and current encoded input, yielding next state.

```

gDec :: S → O → S
gDec (itPrev, →, m, ds, len, param) o = (it, [], newM, newD : ds, newLen, newParam)
where
  (dx, dv, de, dk, dc, ḋ, qtMask, ḟ, utBits) = decodeParts o
  ṙ = if ḋ ≡ 0 ∧ ¬ (null utBits) then head utBits else 0
  newLen = if ṙ ≡ 1 then getInputLen (drop 1 utBits) else len
  m' = if null m then zeros newLen else m
  mtFromH = updateM m' dx dv de dk
  newM = maybe mtFromH id qtMask
  newD = zeros (newLen - length dx) ⊕ reverse dx
  newParam = param

```

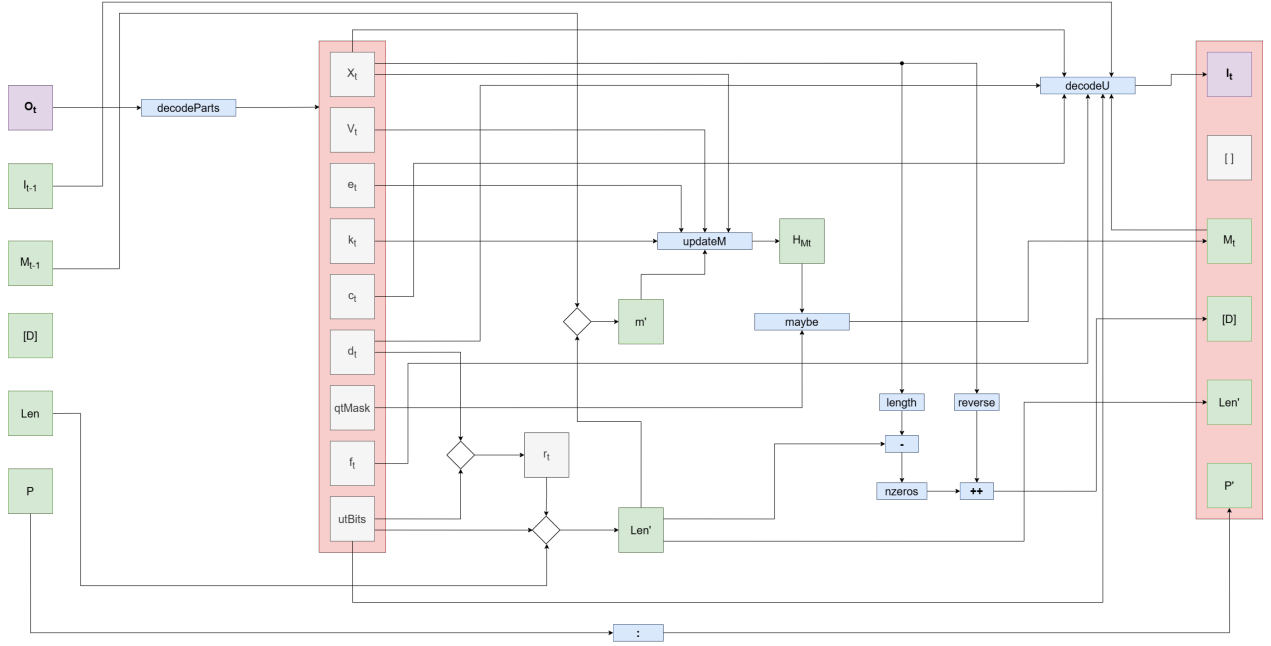


Figure 11: gDec

$$\begin{aligned}
 &\{ \text{sendMask} = \dot{f} : \text{sendMask param} \\
 &\quad , \text{uncompressed} = \dot{r} : \text{uncompressed param} \\
 &\quad \} \\
 &it = \text{decodeU} \dot{d} \text{ dc } \dot{f} \text{ newM } dx \text{ itPrev } utBits
 \end{aligned}$$

f_{Dec} consumes the new state and current encoded input, yielding its reconstruction.

$$\begin{aligned}
 &f_{Dec} :: S \rightarrow O \rightarrow I \\
 &f_{Dec} (itPrev, \rightarrow, \rightarrow, \rightarrow, \rightarrow) o = itPrev
 \end{aligned}$$

Section 5.3.3 Output Vector Components

decodeParts parses the *h*-component of the encoded output and checks the *q*-component to possibly return the new mask.

$$\begin{aligned}
 &\text{decodeParts} :: O \rightarrow (X, \text{Bit}, E, K, C, \text{Bit}, \text{QtMask}, \text{Bit}, [\text{Bit}]) \\
 &\text{decodeParts } o = (dx, dv, de, dk, dc, \dot{d}, qtMask, \dot{f}, utBits) \\
 &\textbf{where} \\
 &\quad (dx, dv, de, dk, dc, \dot{d}, restBits) = \text{decodeH } o \\
 &\quad (qtMask, \dot{f}, utBits) = \text{checkMask } \dot{d} \text{ restBits}
 \end{aligned}$$

Section 5.3.3.1

decodeH is a bit-string parser implemented below using a simple state-monad:

$$\begin{aligned}
 &\text{decodeH} :: [\text{Bit}] \rightarrow (X, \text{Bit}, E, K, C, \text{Bit}, [\text{Bit}]) \\
 &\text{decodeH} = \text{runST } g \textbf{ where}
 \end{aligned}$$

```

g = do { dx ← (decodeXt)  $s_t$ ;
      x ← (nspan 4)  $s_t$ ;
      let dv = fromBinary x
      in do { de ← (decodeE dx dv)  $s_t$ ;
            dk ← (decodeK dx de)  $s_t$ ;
            dc ← (decodeC dk)  $s_t$ ;
             $\dot{d}$  ← (nspan 1)  $s_t$ ;
            rest ← get;
            return (dx, dv, de, dk, dc, head  $\dot{d}$ , rest)
          }
    }

```

To decode each component of h ,

```

decodeXt  $\mathbf{o}_t = (dx, otp)$  where
  dx = unrle  $\mathbf{o}_t$  0
  otp = drop b  $\mathbf{o}_t$ 
  b = length (rle dx)

```

```

decodeE :: X → Bit → [Bit] → (E, [Bit])
decodeE dx dv otp = nspan (if dx ≡ [] ∨ dv ≡ 0 then 0 else 1) otp

```

```

decodeK :: X → E → [Bit] → (K, [Bit])
decodeK dx de otp = nspan (if de ≡ [1] then length dx else 0) otp

```

```

decodeC :: K → [Bit] → (C, [Bit])
decodeC dk otp = nspan (if null dk then 0 else 1) otp

```

Section 5.3.3.2

The next step is to verify if there will be a new mask

```

checkMask :: Bit → [Bit] → (QtMask, Bit, [Bit])
checkMask 1 bits = (Nothing, 0, bits)
checkMask _ [] = (Nothing, 0, [])
checkMask _ (0 : rest) = (Nothing, 0, rest)
checkMask  $\dot{d}$  (f : rest) =
  let (qtPayload, utBits) = splitPayload rest
  fullMask = decodeQ (f : qtPayload)  $\dot{d}$ 
  in (Just fullMask, f, utBits)

```

which will be calculated by

```

decodeQ :: [Bit] → Bit → [Bit]
decodeQ bits  $\dot{d}$ 
  |  $\dot{d} \equiv 1 = bits$ 
  | f ≡ 0 = tail bits -- if f = 0, no more info
  | otherwise = undpcm (reverse (unrle (tail bits) 0))
where
  f = head bits

```

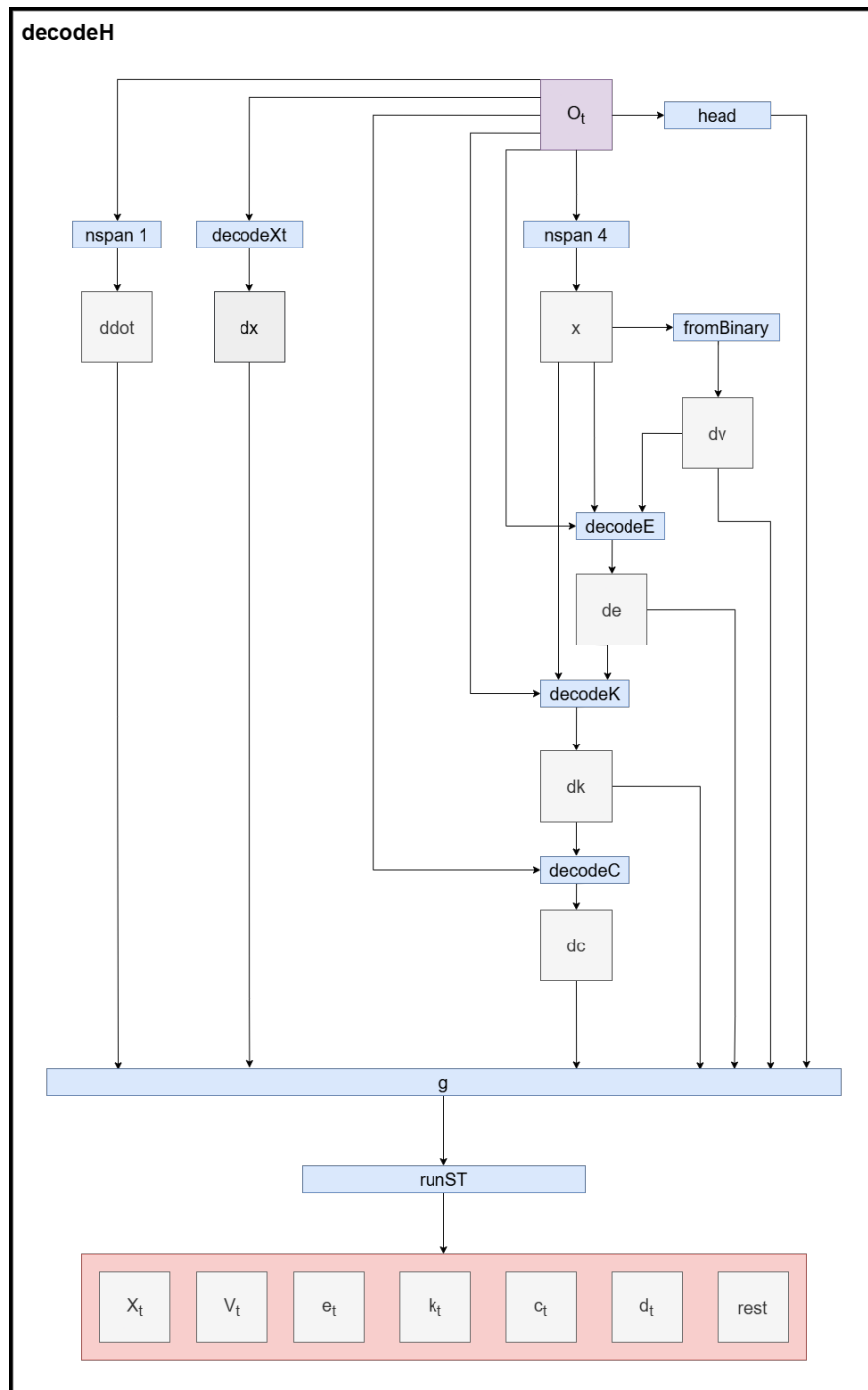


Figure 12: decodeH

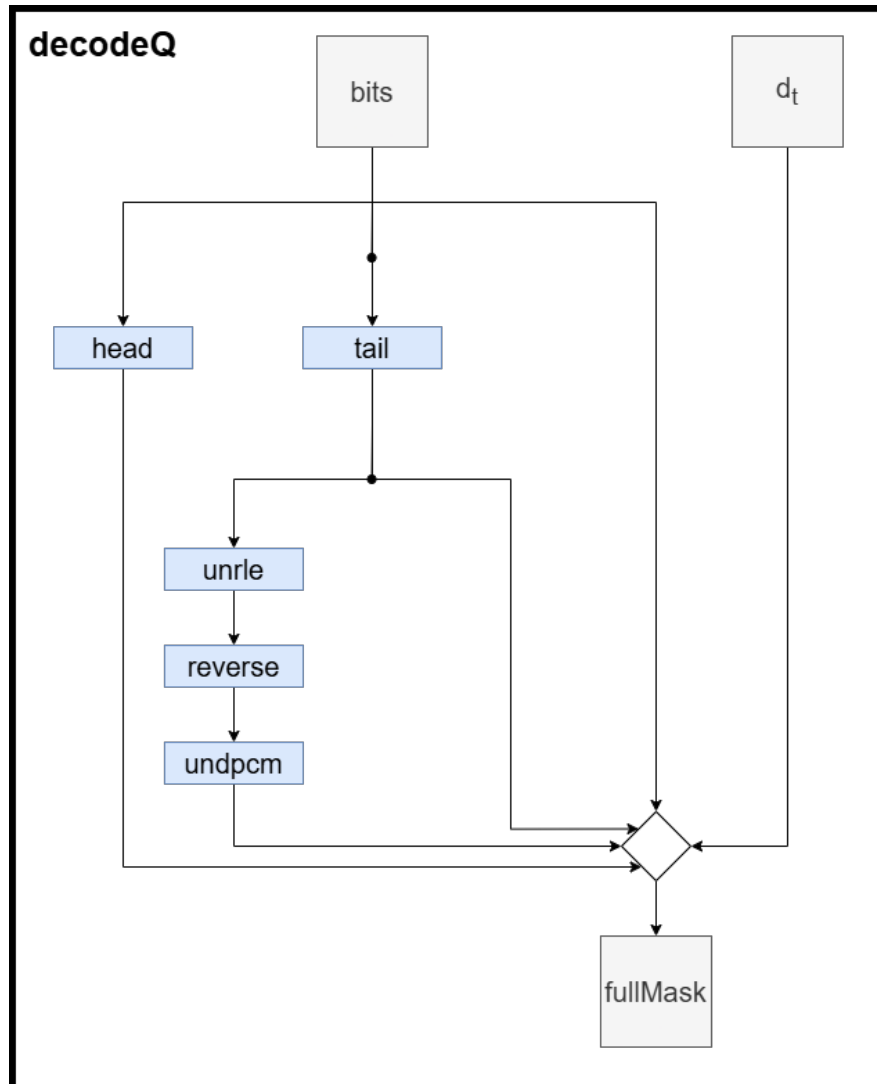


Figure 13: decodeQ

In the first case, the vector only exists if $\dot{d} = 0$. Otherwise, the full mask is the RLE of the reverse of the XOR of M and left-shifted M , so we need to reverse and *undpcm* to get the full mask.

```
undpcm :: [Bit] → [Bit]
undpcm [] = []
undpcm [x] = [x]
undpcm (x : y : xs) = (x ⊕ y) : undpcm (y : xs)
```

This vector does not always encode the full mask, as it is activated by a user-specified flag. With the information obtained from the decoding of the h -component, we can update the mask as follows in the case of no mask encoded by q :

```
updateM :: M → X → Bit → E → K → M
updateM mt dx dv de dk
  | dx ≡ [] = mt
  | dv ≡ 0  = unpredict mt rdx
  | de ≡ [0] = zipWith (λm dv → if dv ≡ 1 then 1 else m) mt rdx
  | de ≡ [1] = fst (mapAccumL aux dk (zip mt rdx))
where
  rdx = nzeros (length mt - length dx) ⊕ reverse dx -- pad to mask length
  aux acc (m, 0) = (acc, m)
  aux (k : ks) (m, 1) = (ks, 1 - k)
  aux [] (m, 1) = ([], m)
```

Section 5.3.3.3

Finally, to recover the input, we need to reconstruct the compression cases defined by f_U .

```
decodeU :: Bit → C → Bit → M → X → I → [Bit] → [Bit]
decodeU dt ct f_t mt xt itPrev utBits
  | dt ≡ 1 = decodeDt1 ct mt xt itPrev utBits
  | otherwise = decodeDt0 f_t ct mt xt itPrev utBits
```

select mask for $\dot{d} = 1$

```
maskDt1 :: C → X → M → M
maskDt1 [1] xt mt = zipWith (∨) (reverse xt) mt
maskDt1 _ _ mt = mt
```

Case 1 and 2: $\dot{d} = 1$

```
decodeDt1 :: C → M → X → I → [Bit] → [Bit]
decodeDt1 ct mt xt itPrev ut =
  let mask = maskDt1 ct xt mt
      bs = reverse (take (sum mask) ut)
  in patch itPrev mask bs
```

select mask for $\dot{d} = 0$

```
maskDt0 :: Bit → C → X → M → M
maskDt0 1 [1] xt mt = zipWith (∨) (reverse xt) mt
maskDt0 _ _ _ mt = mt
```

Case 3, 4 and 5: $\dot{d} = 0$

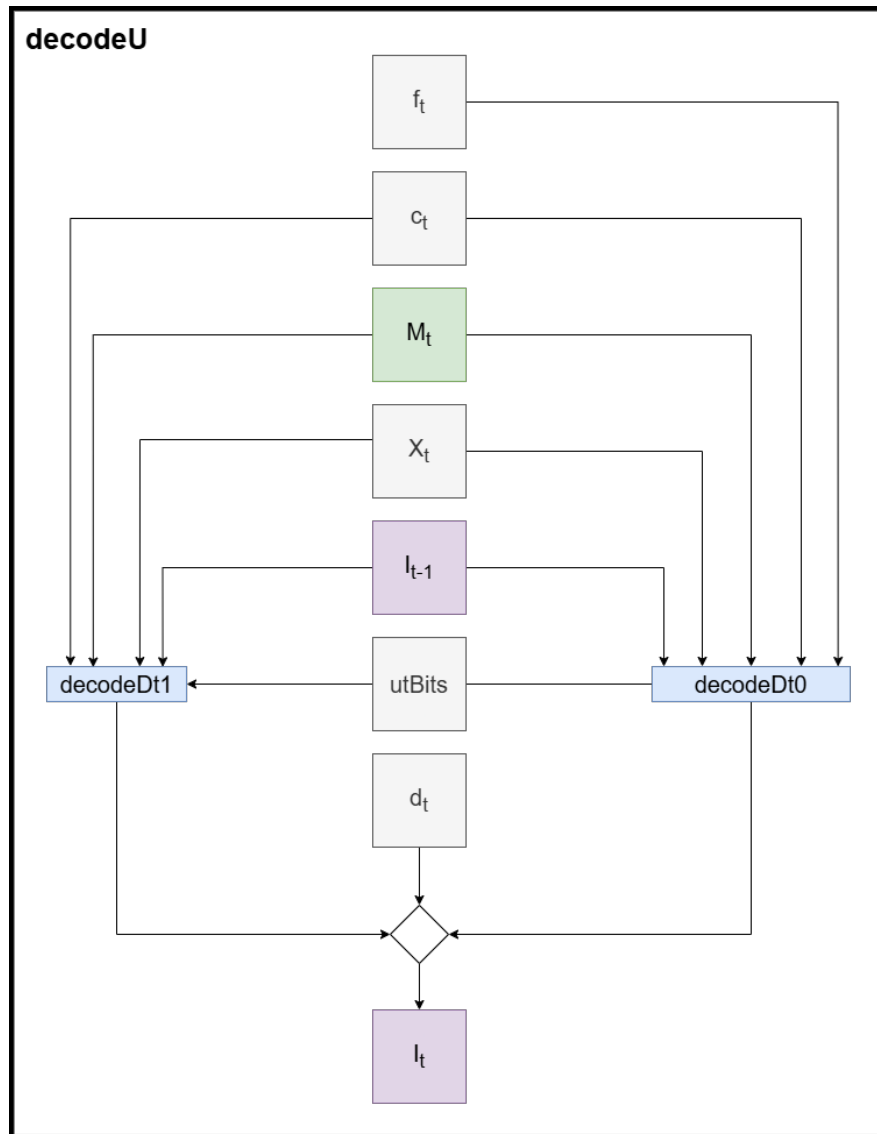


Figure 14: decodeU

```

decodeDt0 :: Bit → C → X → M → I → [Bit] → [Bit]
decodeDt0 _ ct xt mt itPrev bits@(1 : rest) = -- Case 3, Uncompressed (r == 1)
  let f      = getInputLen rest
      payload = drop (length (count f)) rest
      (it, rest') = splitAt f payload
  in it

```

Case 4 and 5: Compressed ($r \equiv 0$)

```

decodeDt0 f _ ct xt mt itPrev (0 : rest) = -- Case 4 and 5: Compressed (r == 0)
  let mask = maskDt0 f _ ct xt mt
      bs = reverse (take (sum mask) rest)
  in 0 : patch itPrev mask bs

```

To get the length of the input in case of uncompressed input, we use:

```

getInputLen :: [Bit] → Int
getInputLen bits = head (uncounting bits)

```

Function to split bits for **q** and **u**:

```

splitPayload :: [Bit] → ([Bit], [Bit])
splitPayload [] = ([], [])
splitPayload [x] = ([x], [])
splitPayload (1 : 0 : xs) = ([1, 0], xs)
splitPayload (x : xs) =
  let (qt, ut) = splitPayload xs
  in (x : qt, ut)

```

RLE

Section 5.2.3 Run-Length Encoding

RLE encoding:

```

rle :: [Bit] → [Bit]
rle = counting · init · map (+1) · map length · splitOn [1]

```

RLE decoding:

```

unrle :: [Bit] → Int → [Bit]
unrle = flip pipeline where
  pipeline n = intercalate [1] · map (flip replicate 0) · map (subtract 1) · trail n · uncounting
  trail n s = s # [n - (sum s + length s)]

```

Section 5.2.2 Counter Encoding Function

Counting:

```

counting :: [Int] → [Bit]
counting = (cGen)

```

where

```

cGen :: Either () (Int, [Bit]) → [Bit]
cGen = [l, r] where
  l = inj10 · nil
  r = conc · (count × id)

```

(6)

The *count* function:

$$\begin{aligned} &count :: Int \rightarrow [Bit] \\ &count\ 1 = in_{j_0}\ [] \\ &count\ a \\ &\quad | a \geq 2 \wedge a \leq 33 = in_{j_{110}}\ (toNBits\ 5\ (a - 2)) \\ &\quad | otherwise = in_{j_{111}}\ (toNBits\ (e\ a)\ (a - 2)) \end{aligned}$$

The 'E' function:

$$\begin{array}{l} e :: \text{Int} \rightarrow \text{Int} \\ e\ a = 2 * \lfloor \log_2 a' + 1 \rfloor - 6 \\ \text{where } a' = \text{fromIntegral } (a - 2) \end{array} \quad (7)$$

NB: the following (simpler) alternative is used instead:

$$\begin{aligned}
& \text{count} :: \text{Int} \rightarrow [\text{Bit}] \\
& \text{count} = [\text{inj}_0 \cdot \text{nil}, g_2] \cdot \text{out}_{\mathbb{N}_0} \cdot \text{pred} \\
& \textbf{where} \\
& \quad g_2 \ a \\
& \quad \quad | \ a \geq 0 \wedge a \leq 31 = \text{inj}_{110} \ (\text{toNBits } 5 \ a) \\
& \quad \quad | \text{otherwise} = \text{inj}_{111} \ (\text{toNBits } (e \ a) \ a) \\
& \quad e \ a = 2 * (\text{minb } a) - 6 \\
& \quad \text{minb } a = \lfloor \log_2(\text{fromIntegral } a) + 1 \rfloor
\end{aligned} \tag{8}$$

Injections:

$$\begin{aligned} inj_0 &= ([0] \#) \\ inj_{10} &= ([1, 0] \#) \\ inj_{110} &= ([1, 1, 0] \#) \\ inj_{111} &= ([1, 1, 1] \#) \end{aligned} \tag{9}$$

Uncounting:

$$\begin{aligned} \text{uncounting} &:: [\text{Bit}] \rightarrow [\text{Int}] \\ \text{uncounting} &= [(parse)] \end{aligned} \quad (10)$$

where the bit vector 'parsing gene' is:

$$\begin{aligned}
& \text{parse } [1, 0] = \text{Nothing} \\
& \text{parse } (0 : r) = \text{Just } (1, r) \\
& \text{parse } (1 : (1 : (0 : r))) = \text{Just } (\text{fromBinary } a + 2, r') \\
& \quad \textbf{where } (a, r') = \text{nspan } 5 \ r \\
& \text{parse } (1 : (1 : (1 : r))) = \text{Just } (\text{fromBinary } a + 2, r') \\
& \quad \textbf{where } (a, r') = \text{nspan } (6 + \text{length } x) \ r''; (x, r'') = \text{cspan } (\equiv 0) \ r \\
& \text{parse } _ = \text{Nothing}
\end{aligned} \tag{11}$$

pocket.hs

To test the encoder and decoder, we use the following test vectors, taken from the reference implementation in C++. The *bits* vector is the input to be compressed, *compressed* is the output of the encoder, and *decompressed* is the output of the decoder, which should match *bits*.

```
bits :: [[Bit]]
bits = [[1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1], [1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]]
```

[illegible]

The test can be done automatically with:

```
testEncoder :: Bool
testEncoder = compressed ≡ snd (encoder bits)

testDecoder :: Bool
testDecoder = decompressed ≡ snd (decoder compressed)
```

ENCODER : Testing specific states and inputs

We can also test specific states and inputs, to verify the intermediate calculations and outputs of the encoder. To do this, we define each state and output vector separately, as follows.

$$\begin{aligned}
i_0 &= [1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1] \\
i_1 &= [1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1] \\
i_2 &= [1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1] \\
i_3 &= [1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0] \\
i_4 &= [1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0] \\
i_5 &= [1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0] \\
i_6 &= [1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1] \\
s_{ie} &= ([], [], [], [], 32, \text{initParamEnc}) \\
s_{0e} &= g_{Enc} \ s_{ie} \ i_0 \\
e_0 &= f_{Enc} \ s_{0e} \ i_0 \\
s_{1e} &= g_{Enc} \ s_{0e} \ i_1 \\
e_1 &= f_{Enc} \ s_{1e} \ i_1 \\
s_{2e} &= g_{Enc} \ s_{1e} \ i_2 \\
e_2 &= f_{Enc} \ s_{2e} \ i_2 \\
s_{3e} &= g_{Enc} \ s_{2e} \ i_3 \\
e_3 &= f_{Enc} \ s_{3e} \ i_3 \\
s_{4e} &= g_{Enc} \ s_{3e} \ i_4 \\
e_4 &= f_{Enc} \ s_{4e} \ i_4 \\
s_{5e} &= g_{Enc} \ s_{4e} \ i_5 \\
e_5 &= f_{Enc} \ s_{5e} \ i_5 \\
s_{6e} &= g_{Enc} \ s_{5e} \ i_6 \\
e_6 &= f_{Enc} \ s_{6e} \ i_6
\end{aligned}$$

DECODER : Testing specific states and inputs

To test specific states and inputs for the decoder, we can use the same states as for the encoder, but now as input to the decoder, and the output should match the input of the encoder.

$$\begin{aligned} o_0 &= [1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1] \\ o_1 &= [1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1] \\ o_2 &= [1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1] \\ o_3 &= [0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1] \\ o_4 &= [0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1] \\ o_5 &= [0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1] \end{aligned}$$

```

o6 = [1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1]
sid = ([], [], [], [], 0, initParamDec)
s0d = gDec sid o0
d0 = fDec s0d o0
s1d = gDec s0d o1
d1 = fDec s1d o1
s2d = gDec s1d o2
d2 = fDec s2d o2
s3d = gDec s2d o3
d3 = fDec s3d o3
s4d = gDec s3d o4
d4 = fDec s4d o4
s5d = gDec s4d o5
d5 = fDec s5d o5
s6d = gDec s5d o6
d6 = fDec s6d o6

```

Auxiliary functions

The making of a Mealy machine:³

```

mealy :: ((s, i) → s) → ((s, i) → o) → (s, i) → (s, o)
mealy g f = ⟨fst, f⟩ · ⟨g, snd⟩

```

Generating 0-vectors:

```

zeros = map 0
nzeros n = replicate n 0

```

Bit (un)predictability:

```

unpredict = zipWith ⊕

```

Encode positive integer in specified number of bits:⁴

```

toNBits :: Int → Int → [Bit]
toNBits n x = nzeros (n - length bits) # bits
  where
    bits = toBinary x

```

Converse of toNBits:

```

fromNBits :: Int → [Bit] → Int
fromNBits _ = fromBinary

```

Auxiliary, for non-negative integers:

```

toBinary :: Int → [Bit]
toBinary 0 = [0]
toBinary x = reverse (helper x)

```

³Note how *g* updates the state before *f* yields the output.

⁴Cf. the alternative *toNBits e = reverse · take e · [(Just · swap · flip divMod 2)]*.

where

helper 0 = []
helper y = (y 'mod' 2) : *helper* (y 'div' 2)

Note the post-condition:

$$x > 0 \Rightarrow \text{length } (\text{toBinary } x) = \lfloor \log_2 x \rfloor + 1. \quad (12)$$

Converse of *toBinary*:

fromBinary :: [Bit] → Int
fromBinary = foldl (\acc b → acc * 2 + b) 0

Parameter rotation:

next :: Param → Param
next (Param m nm rl u s) = Param m (rot nm) (rot rl) (rot u) (rot s)
where rot (h : t) = t # [h] -- non-empty list rotation

Auxiliary function to patch the recovered input:

patch :: I → M → [Bit] → I
patch [] _ _ = []
patch (p : ps) [] _ = p : ps
patch (p : ps) (m : ms) [] = p : *patch* ps ms []
patch (p : ps) (m : ms) (b : bs)
 | m ≡ 1 = b : *patch* ps ms bs
 | otherwise = p : *patch* ps ms (b : bs)

Span on count:

$$\text{nspan } n = \langle \text{take } n, \text{drop } n \rangle \quad (13)$$

Span on condition:

$$\text{cspan } p = \langle \text{takeWhile } p, \text{dropWhile } p \rangle \quad (14)$$

Basic properties

Let us define

$$|n| = (\equiv n) \cdot \text{length} \quad (15)$$

$$\lceil n \rceil = (\in \{0 \dots n-1\}) \quad (16)$$

Then *toNBits* *n* has relational type $[2^n] \rightarrow |n|$, that is,

$$\text{toNBits } n \cdot \Phi_{\lceil 2^n \rceil} \subseteq \Phi_{|n|} \cdot \text{toNBits } n \quad (17)$$

Moreover, we have the following basic properties:

$$\text{fromBinary} \cdot \text{toNBits } n = \text{id} \quad (18)$$

$$\text{length} \cdot (\text{flip replicate } a) = \text{id} \quad (19)$$

$$\text{intercalate } a \cdot (\text{splitOn } a \ x) = x \quad (20)$$

$$(\text{subtract } 1) \cdot (+1) = \text{id} \quad (21)$$

$$\text{init } (\text{trail } n \ xs) = xs \quad (22)$$

Despite the functions not always being direct inverses of each other, these properties are enough to show that *unrle* · *rle* = *id*, as the pipeline of *rle* can now be inverted step by step.

It remains to verify the properties of *count* and *uncounting*. We will get to them in the next section.

From the associative property of concatenation, $a \# (b \# c) = (a \# b) \# c$, for *inj* any of the injections of (9), we have

$$\text{conc} \cdot (\text{inj} \times \text{id}) = \text{inj} \cdot \text{conc} \quad (23)$$

Concerning *conc*, *nspan* (13) and *cspan*:

$$\text{conc} \cdot \text{nspan } n = \text{id} \quad (24)$$

$$\text{conc} \cdot \text{cspan } p = \text{id} \quad (25)$$

$$\widehat{\text{nspan}} \cdot \langle \text{length} \cdot \pi_1, \text{conc} \rangle = \text{id} \quad (26)$$

Property (26) says that $\text{length} \cdot \pi_1$ is a least complement of *conc*. Another way to say the same is:

$$\text{nspan } n \cdot \text{conc} \cdot (\Phi_{|n|} \times \text{id}) \subseteq \text{id} \quad (27)$$

Correctness argument details

To assist the correctness, the following rules of relation algebra [?] will be used:

Involution	$(R^\circ)^\circ = R$	(28)
Contravariance	$(R \cdot S)^\circ = S^\circ \cdot R^\circ$	(29)
Shunting Rules	$R \cdot f^\circ \subseteq S \equiv R \subseteq S \cdot f$	(30)
	$f \cdot R \subseteq S \equiv R \subseteq f^\circ \cdot S$	(31)
Relation Union Universal property	$R \cup S \subseteq X \equiv R \subseteq X \wedge S \subseteq X$	(32)
Relation Union Right Linearity	$R \cdot (S \cup Q) = R \cdot S \cup R \cdot Q$	(33)
Converse-$\cup$	$(R \cup S)^\circ = R^\circ \cup S^\circ$	(34)
Converse-coreflexive	$\Phi_p^\circ = \Phi_p$	(35)
Guards	$p? = [\Phi_p, \Phi_{\neg p}]^\circ$	(36)
McCarthy conditional	$p \rightarrow R, S = [R, S] \cdot p?$	(37)
Functor-$\times$-composition	$(R \times S) \cdot (P \times Q) = (R \cdot P) \times (S \cdot Q)$	(38)
Functor-$\times$-converse	$(R \times S)^\circ = R^\circ \times S^\circ$	(39)
Coproduct Definition	$[R, S] = R \cdot i_1^\circ \cup S \cdot i_2^\circ$	(40)
+fusion	$R \cdot [S, Q] = [R \cdot S, R \cdot Q]$	(41)
Divide and Conquer	$[R, S] \cdot [Q, U]^\circ = (R \cdot Q^\circ) \cup (S \cdot U^\circ)$	(42)
Extensional Equality	$f = g \Leftrightarrow \langle \forall x :: f x = g x \rangle$	(43)
Def-$\times$	$(f \times g)(a, b) = (f a, g b)$	(44)
Def-split	$\langle f, g \rangle x = (f x, g x)$	(45)
Cata-cancellation	$\langle \alpha \rangle = \alpha \cdot F \langle \alpha \rangle \cdot \text{out}$	(46)

Calculation of *uncounting* (10) The strategy is to invert *counting* (6) in order to obtain *Uncounting* (a relation, in general — hopefully function (10) in the end):

$$\begin{aligned}
& \text{Uncounting} = \text{counting}^\circ \\
\equiv & \quad \{ \text{definition of } \text{counting} \} \\
& \text{Uncounting} = \llbracket cGen \rrbracket^\circ \\
\equiv & \quad \{ (46), (29), (28) \} \\
& \text{Uncounting} = \llbracket (cGen^\circ) \rrbracket \\
\equiv & \quad \{ \text{let } Parse = cGen^\circ \text{ — a non-deterministic parser to begin with} \} \\
& \text{Uncounting} = \llbracket Parse \rrbracket
\end{aligned}$$

counting, a catamorphism, is illustrated in (47) and *Uncounting*, an anamorphism, is illustrated in (48).

$$\begin{array}{ccc}
\mathbb{N}_0^* & \xrightarrow{\text{out}_{\mathbb{N}_0}} & 1 + \mathbb{N}_0 \times \mathbb{N}_0^* \\
\llbracket cGen \rrbracket \downarrow & & \downarrow \text{id} + \text{id} \times \llbracket cGen \rrbracket \\
Bit^* & \xleftarrow{cGen} & 1 + \mathbb{N}_0 \times Bit^*
\end{array} \tag{47}$$

$$\begin{array}{ccc}
\mathbb{N}_0^* & \xleftarrow{\text{in}_{\mathbb{N}_0}} & 1 + \mathbb{N}_0 \times \mathbb{N}_0^* \\
\llbracket (cGen^\circ) \rrbracket \uparrow & & \uparrow \text{id} + \text{id} \times \llbracket (cGen^\circ) \rrbracket \\
Bit^* & \xrightarrow{cGen^\circ} & 1 + \mathbb{N}_0 \times Bit^*
\end{array} \tag{48}$$

Unfolding *Parse*:

$$\begin{aligned}
& Parse = cGen^\circ \\
\equiv & \quad \{ \text{definition of } cGen; (40); (34); (29) \times 2 \} \\
& Parse = i_1 \cdot l^\circ \cup i_2 \cdot r^\circ \\
\equiv & \quad \{ \text{definition of } cGen; (29) \times 2; (39) \} \\
& Parse = i_1 \cdot nil^\circ \cdot inj_{10}^\circ \cup i_2 \cdot (count^\circ \times id) \cdot conc^\circ
\end{aligned}$$

Inverting *count*:

$$\begin{aligned}
& count = [inj_0 \cdot nil, g_2] \cdot out_{\mathbb{N}_0} \cdot pred \\
\equiv & \quad \{ (29) \times 2 \} \\
& count^\circ = succ \cdot in_{\mathbb{N}_0} \cdot [inj_0 \cdot nil, g_2]^\circ \\
\equiv & \quad \{ in_{\mathbb{N}_0} = [\underline{0}, succ]; 41 \} \\
& count^\circ = [\underline{1}, (2+)] \cdot [inj_0 \cdot nil, g_2]^\circ \\
\equiv & \quad \{ 42 \} \\
& count^\circ = \underline{1} \cdot (inj_0 \cdot nil)^\circ \cup (2+) \cdot g_2^\circ
\end{aligned}$$

Abbreviating

$$tnb\ f = \overline{toNBits} \cdot \langle f, id \rangle \quad (49)$$

$$pre = \lceil 2^5 \rceil \quad (50)$$

$$post = \lfloor 5 \rfloor \quad (51)$$

one has:⁵

$$\begin{aligned} g_2 &= pre \rightarrow inj_{110} \cdot tnb\ \underline{5}, inj_{111} \cdot tnb\ e \\ &\equiv \{ 37; 36; 42; 35 \} \\ g_2 &= inj_{110} \cdot (tnb\ \underline{5} \cdot \Phi_{pre}) \cup inj_{111} \cdot (tnb\ e \cdot \Phi_{\neg pre}) \\ &\equiv \{ 34; 29 \} \\ g_2^\circ &= (tnb\ \underline{5} \cdot \Phi_{pre})^\circ \cdot inj_{110}^\circ \cup (tnb\ e \cdot \Phi_{\neg pre})^\circ \cdot inj_{111}^\circ \\ &\Rightarrow \{ 33 \} \\ (2+) \cdot g_2^\circ &= (2+) \cdot (tnb\ \underline{5} \cdot \Phi_{pre})^\circ \cdot inj_{110}^\circ \cup (2+) \cdot (tnb\ e \cdot \Phi_{\neg pre})^\circ \cdot inj_{111}^\circ \end{aligned}$$

Putting everything together:

$$count^\circ = \begin{cases} \underline{1} \cdot (inj_0 \cdot nil)^\circ \\ \cup (2+) \cdot (tnb\ \underline{5} \cdot \Phi_{pre})^\circ \cdot inj_{110}^\circ \\ \cup (2+) \cdot (tnb\ e \cdot \Phi_{\neg pre})^\circ \cdot inj_{110}^\circ \end{cases}$$

Moreover ($\cup$ -distribution, etc):

$$\begin{aligned} &i_2 \cdot (count^\circ \times id) \cdot conc^\circ \\ &= \{ \text{definition of count; 33; 29} \} \\ &\left\{ \begin{array}{l} i_2 \cdot (\underline{1} \cdot (inj_0 \cdot nil)^\circ \times id) \cdot conc^\circ \\ \cup i_2 \cdot ((2+) \cdot (tnb\ \underline{5} \cdot \Phi_{pre})^\circ \times id) \cdot (conc \cdot (inj_{110} \times id))^\circ \\ \cup i_2 \cdot ((2+) \cdot (tnb\ e \cdot \Phi_{\neg pre})^\circ \times id) \cdot (conc \cdot (inj_{111} \times id))^\circ \end{array} \right. \\ &= \{ id \cdot (id \cdot id) = id; 38; 29; 28; 39; id^\circ = id; 23 \} \\ &\left\{ \begin{array}{l} i_2 \cdot (\underline{1} \times id) \cdot (conc \cdot (inj_0 \times id) \cdot (nil \times id))^\circ \\ \cup i_2 \cdot ((2+) \cdot (tnb\ \underline{5} \cdot \Phi_{pre})^\circ \times id) \cdot (inj_{110} \cdot conc)^\circ \\ \cup i_2 \cdot ((2+) \cdot (tnb\ e \cdot \Phi_{\neg pre})^\circ \times id) \cdot (inj_{111} \cdot conc)^\circ \end{array} \right. \end{aligned}$$

Finally:

$$Parse = \begin{cases} i_1 \cdot nil^\circ \cdot inj_{10}^\circ \\ \cup i_2 \cdot (\underline{1} \times id) \cdot (inj_0 \cdot conc \cdot (nil \times id))^\circ \\ \cup i_2 \cdot ((2+) \cdot (tnb\ \underline{5} \cdot \Phi_{pre})^\circ \times id) \cdot (inj_{110} \cdot conc)^\circ \\ \cup i_2 \cdot ((2+) \cdot (tnb\ e \cdot \Phi_{\neg pre})^\circ \times id) \cdot (inj_{111} \cdot conc)^\circ \end{cases}$$

⁵Also note, if needed: $(2+) \cdot \Phi_{pre} = \Phi_{post} \cdot (2+)$ where $post\ a = a \in [2 \dots 33]$.

Calculating *parse*. The equality above can be factored into a $\subseteq$ -part and a $\supseteq$ -part. Let us focus on the latter:

$$\begin{aligned}
& \left\{ \begin{array}{l} i_1 \cdot \text{nil}^\circ \cdot \text{inj}_{10}^\circ \\ \cup \quad i_2 \cdot (\underline{1} \times \text{id}) \cdot (\text{inj}_0 \cdot \text{conc} \cdot (\text{nil} \times \text{id}))^\circ \\ \cup \quad i_2 \cdot ((2+) \cdot (\text{tnb } \underline{5} \cdot \Phi_{pre})^\circ \times \text{id}) \cdot (\text{inj}_{110} \cdot \text{conc})^\circ \\ \cup \quad i_2 \cdot ((2+) \cdot (\text{tnb } e \cdot \Phi_{\neg pre})^\circ \times \text{id}) \cdot (\text{inj}_{111} \cdot \text{conc})^\circ \end{array} \right\} \subseteq \text{Parse} \\
& \equiv \quad \{ 32; 30; 29 \} \\
& \left\{ \begin{array}{l} i_1 \cdot \text{nil}^\circ \subseteq \text{Parse} \cdot \text{inj}_{10} \\ i_2 \cdot (\underline{1} \times \text{id}) \cdot \pi_2^\circ \subseteq \text{Parse} \cdot \text{inj}_0 \\ i_2 \cdot ((2+) \cdot (\text{tnb } \underline{5} \cdot \Phi_{pre})^\circ \times \text{id}) \cdot \text{conc}^\circ \subseteq \text{Parse} \cdot \text{inj}_{110} \\ i_2 \cdot ((2+) \cdot (\text{tnb } e \cdot \Phi_{\neg pre})^\circ \times \text{id}) \cdot \text{conc}^\circ \subseteq \text{Parse} \cdot \text{inj}_{111} \end{array} \right\} \quad (52)
\end{aligned}$$

It is already clear that *Parse* is doing ‘induction’ on the four bit injections (9). Moving on:

$$\begin{aligned}
& (52) \\
& \equiv \quad \{ 30 \} \\
& \left\{ \begin{array}{l} i_1 \subseteq \text{Parse} \cdot \text{inj}_{10} \cdot \text{nil} \\ i_2 \cdot (\underline{1} \times \text{id}) \subseteq \text{Parse} \cdot \text{inj}_0 \cdot \pi_2 \\ i_2 \cdot ((2+) \cdot (\text{tnb } \underline{5} \cdot \Phi_{pre})^\circ \times \text{id}) \subseteq \text{Parse} \cdot \text{inj}_{110} \cdot \text{conc} \\ i_2 \cdot ((2+) \cdot (\text{tnb } e \cdot \Phi_{\neg pre})^\circ \times \text{id}) \subseteq \text{Parse} \cdot \text{inj}_{111} \cdot \text{conc} \end{array} \right\} \\
& \Leftarrow \quad \{ (18) ; \text{assume some } post' \text{ such that } \text{tnb } e : \neg pre \rightarrow post' \} \\
& \left\{ \begin{array}{l} i_1 \subseteq \text{Parse} \cdot \text{inj}_{10} \cdot \text{nil} \\ i_2 \cdot (\underline{1} \times \text{id}) \subseteq \text{Parse} \cdot \text{inj}_0 \cdot \pi_2 \\ i_2 \cdot ((2+) \cdot \text{fromBinary} \times \text{id}) \cdot (\Phi_{post} \times \text{id}) \cdot \text{conc}^\circ \subseteq \text{Parse} \cdot \text{inj}_{110} \\ i_2 \cdot ((2+) \cdot \text{fromBinary} \times \text{id}) \cdot (\Phi_{post'} \times \text{id}) \cdot \text{conc}^\circ \subseteq \text{Parse} \cdot \text{inj}_{111} \end{array} \right\} \\
& \Leftarrow \quad \{ \text{raise lower side by (27) + introduce } f \text{ (see below)} \} \\
& \left\{ \begin{array}{l} i_1 \subseteq \text{Parse} \cdot \text{inj}_{10} \cdot \text{nil} \\ i_2 \cdot (\underline{1} \times \text{id}) \subseteq \text{Parse} \cdot \text{inj}_0 \cdot \pi_2 \\ i_2 \cdot ((2+) \cdot \text{fromBinary} \times \text{id}) \cdot (\text{nspan } 5) \subseteq \text{Parse} \cdot \text{inj}_{110} \\ i_2 \cdot ((2+) \cdot \text{fromBinary} \times \text{id}) \cdot f \subseteq \text{Parse} \cdot \text{inj}_{111} \end{array} \right\}
\end{aligned}$$

Since only functions are involved just above, it makes sense to further strengthen *Parse* to a function *parse*, making it a deterministic parser:

$$\begin{aligned}
& \left\{ \begin{array}{l} \text{parse} \cdot \text{inj}_{10} \cdot \text{nil} = i_1 \\ \text{parse} \cdot \text{inj}_0 \cdot \pi_2 = i_2 \cdot (\underline{1} \times \text{id}) \\ \text{parse} \cdot \text{inj}_{110} \cdot \text{nspan } 5 = i_2 \cdot ((2+) \cdot \text{fromBinary} \times \text{id}) \\ \text{parse} \cdot \text{inj}_{111} \cdot f = i_2 \cdot ((2+) \cdot \text{fromBinary} \times \text{id}) \end{array} \right\} \\
& \equiv \quad \{ 43; \text{definitions of } \text{inj}_{10} \text{ and } \text{inj}_0; 44; 45 \} \\
& \left\{ \begin{array}{l} \text{parse } [1, 0] = i_1 () \\ \text{parse } (0 : r) = i_2 (1, r) \\ \text{parse } (\text{inj}_{110} r) = i_2 (\text{fromBinary } a + 2, r') \text{ where } (a, r') = \text{nspan } 5 r \\ \text{parse } (\text{inj}_{111} r) = i_2 (\text{fromBinary } a + 2, y) \text{ where } (a, y) = f r \end{array} \right\}
\end{aligned}$$

What remains to find is *post'* and *f* such that

$$\begin{aligned}
& \text{tnb } e : \neg [2^5] \rightarrow post' \\
& f \cdot \text{conc} \cdot (\Phi_{post'} \times \text{id}) \subseteq \text{id}
\end{aligned}$$

Here resides one of the trickiest parts of the whole *RLE* encoding, to be found in the ‘E’ function (8). Looking

at the definition, we get

$$e\ a + 6 = 2 * (\text{minb } a)$$

where $\text{minb } a$ is the minimum number of bits needed to encode a , which always start with a 1, for $a \neq 0$. Thus $\text{tnb } e\ a$ has $\text{minb } a - 6$ leading zeros. So, counting them and adding 6 gives us $\text{minb } a$. In fact:

$$\begin{aligned} & \widehat{\text{toNBits}} \cdot \langle e, id \rangle \\ = & \{ \text{uncurrying} \} \\ & \text{toNBits } (e\ a)\ a \\ = & \{ \text{definition of toNBits} \} \\ & \text{toNBits } (e\ a)\ a = \text{nzeros } ((e\ a) - \text{length bits}) \# \text{bits where bits} = \text{toBinary } a \\ = & \{ \text{length } (\text{toBinary } a) = \text{minb } a \} \\ & \text{toNBits } (e\ a)\ a = \text{nzeros } ((e\ a) - \text{minb } a) \# \text{toBinary } a \\ = & \{ \text{definition of } e \} \\ & \text{toNBits } (e\ a)\ a = \text{nzeros } (2 (\text{minb } a) - 6 - \text{minb } a) \# \text{toBinary } a \\ = & \{ \text{definition (49)} \} \\ & \text{tnb } e = \text{conc} \cdot \langle \text{nzeros} \cdot h, \text{toBinary} \rangle \text{ where } h\ a = \text{minb } a - 6 \end{aligned}$$

Thus:

$$\text{tnb } e \cdot \Phi_{\neg \text{pre}} \subseteq \text{conc} \cdot \langle \text{nzeros} \cdot h, \text{toBinary} \rangle \text{ where } h\ a = \text{minb } a - 6 \quad (53)$$

Then:

$$\begin{aligned} & i_2 \cdot ((2+) \cdot (\text{tnb } e \cdot \Phi_{\neg \text{pre}})^\circ \times id) \subseteq \text{Parse} \cdot \text{inj}_{111} \cdot \text{conc} \\ \Leftarrow & \{ (53) \} \\ & i_2 \cdot ((2+) \times id) \cdot (\text{conc} \cdot \langle \text{nzeros} \cdot h, \text{toBinary} \rangle \times id)^\circ \subseteq \text{Parse} \cdot \text{inj}_{111} \cdot \text{conc} \\ \equiv & \{ 30 \} \\ & i_2 \cdot ((2+) \times id) \subseteq \text{Parse} \cdot \text{inj}_{111} \cdot \text{conc} \cdot (\text{conc} \times id) \cdot (\langle \text{nzeros} \cdot h, \text{toBinary} \rangle \times id) \\ \equiv & \{ \text{associate to the right} \} \\ & i_2 \cdot ((2+) \times id) \subseteq \text{Parse} \cdot \text{inj}_{111} \cdot \text{conc} \cdot \langle \text{nzeros} \cdot h \cdot \pi_1, \text{conc} \cdot (\text{toBinary} \times id) \rangle \end{aligned}$$

It is clear from above that $\text{nzeros} \cdot h \cdot \pi_1$ is a form of encoding the length of the bit string generated by toBinary , using zeros to represent such a number in binary. Lack of time prevents us from reaching

$$i_2 \cdot ((2+) \cdot \text{fromBinary} \times id) \cdot \widehat{\text{nspan}} \cdot ((6+) \cdot \text{length} \times id) \cdot \text{cspan} (\equiv 0) \subseteq \text{Parse} \cdot \text{inj}_{111}$$

wherefrom

$$f = \widehat{\text{nspan}} \cdot ((6+) \cdot \text{length} \times id) \cdot \text{cspan} (\equiv 0)$$

can be calculated, finally leading to

$$\begin{aligned} & \text{parse } [1, 0] = \text{Nothing} \\ & \text{parse } (0 : r) = \text{Just } (1, r) \\ & \text{parse } (1 : (1 : (0 : r))) = \text{Just } (\text{fromBinary } a + 2, r') \\ & \quad \text{where } (a, r') = \text{nspan } 5\ r \\ & \text{parse } (1 : (1 : (1 : r))) = \text{Just } (\text{fromBinary } a + 2, r') \\ & \quad \text{where } (a, r') = \text{nspan } (6 + \text{length } x)\ r''; (x, r'') = \text{cspan } (\equiv 0)\ r \\ & \text{parse } _ = \text{Nothing} \end{aligned}$$

where i_1 and i_2 are transliterated to the *Maybe* data type, as required by Haskell's *unfoldr* combinator used in (10). Formally calculating this last step of the exercise is not immediate and is left for future work.

xxx Missing part:

takeWhile ($\neq 1$) $\cdot minb = nil$ for positive inputs.

TBC